Lecture 1

CME 295: Transformers & Large Language Models





Afshine Amidi & Shervine Amidi



Stanford University

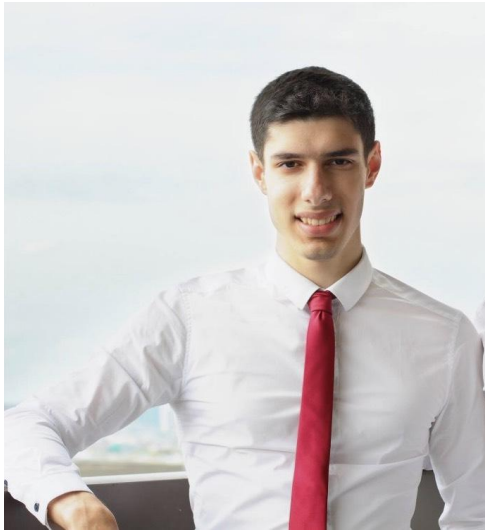
ICME

Teaching staff



Afshine and **Shervine**

Teaching staff



Afshine
Centrale Paris ('16), MIT ('17)
Uber, Google, Netflix



Shervine
Centrale Paris ('16), Stanford ('19)
Uber, Google, Netflix

Welcome to CME 295!

Goals.

1. Understand how **Transformers** work and how they **relate** to **LLMs**
2. Learn how **LLMs** are **trained** and **used** in various **applications**

Welcome to CME 295!

Goals.

1. Understand how **Transformers** work and how they **relate** to **LLMs**
2. Learn how **LLMs** are **trained** and **used** in various **applications**

Audience.

- Interested in LLMs
  - Career goal
  - Personal project
  - "AI literacy" or curiosity
- Prerequisite: machine learning basics, linear algebra

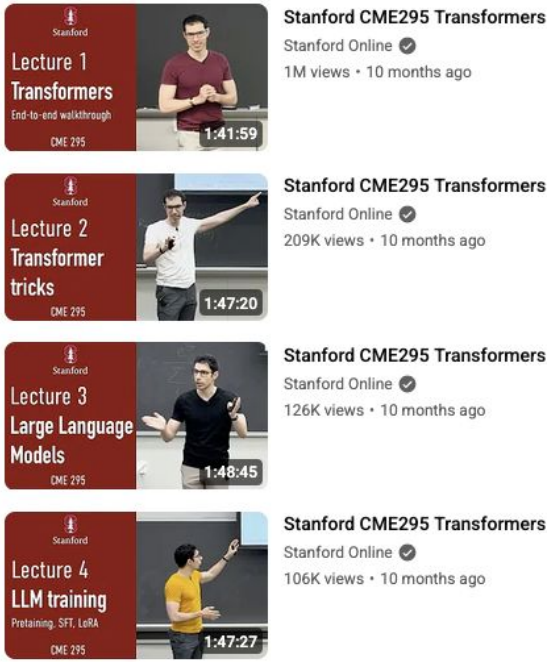
Difference with last year's edition

New this year.

- Post-training approaches
- AI Agents
- Diffusion LLMs

...among other things!





Last year's playlist

Logistics

Date & time.

- Fridays from 3:30pm to 5:20pm
- Thornton 110

Logistics

Date & time.

- Fridays from 3:30pm to 5:20pm
- Thornton 110

Details about the class.

- 2 units
- Letter or Credit/No credit
- Lectures are recorded
- Midterm (50% grade), scheduled on Friday, October 23rd
- Final exam (50% grade), scheduled on Wednesday, December 9th

Material

Class website. cme295.stanford.edu

- Contains syllabus & logistics
- Recordings will be posted there
- Slides available day before lecture

Material

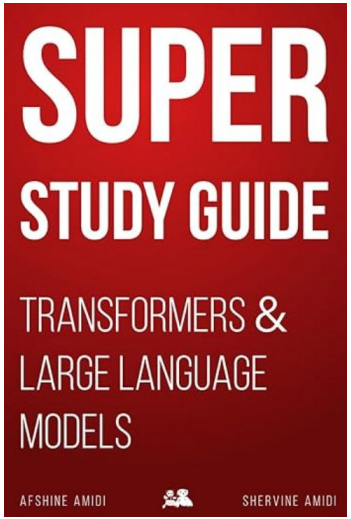
Class website. cme295.stanford.edu

- Contains syllabus & logistics
- Recordings will be posted there
- Slides available day before lecture

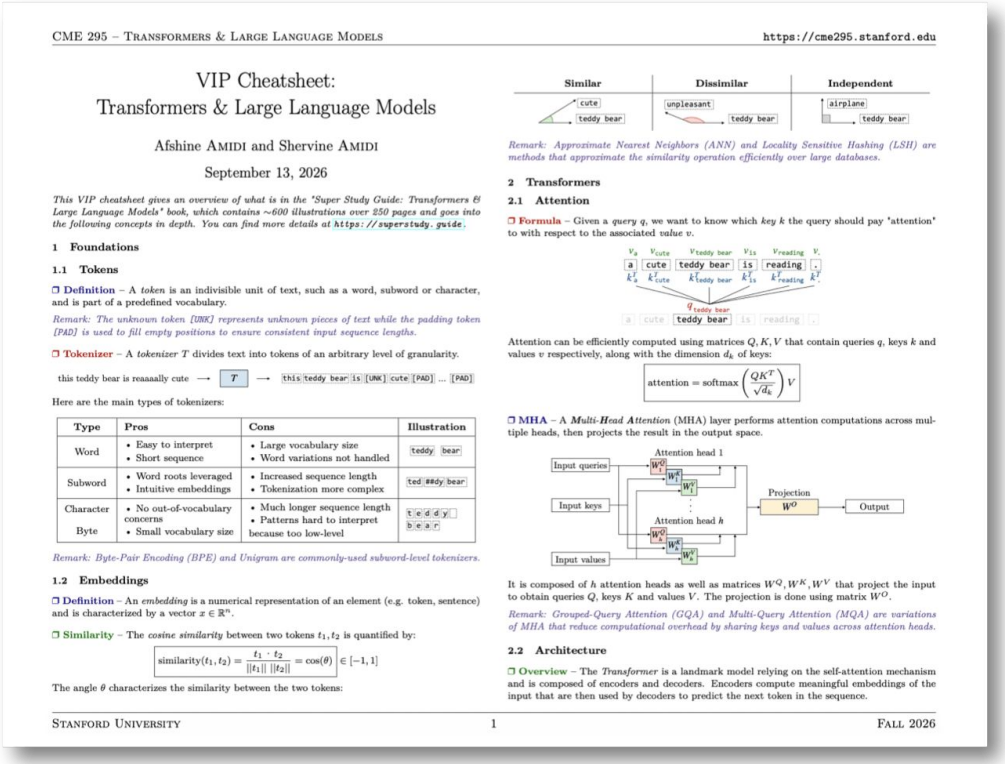
Class textbook.

Super Study Guide:
Transformers &
Large Language Models

Book: https://superstudy.guide



Material



CME 295 – Transformers & Large Language Models — https://cme295.stanford.edu

VIP Cheatsheet: Transformers & Large Language Models

Afshine Amidi and Shervine Amidi

September 13, 2026

This VIP cheatsheet gives an overview of what is in the "Super Study Guide: Transformers & Large Language Models" book, which contains ~600 illustrations over 250 pages and goes into the following concepts in depth. You can find more details at `https://superstudy.guide`.

1 Foundations

1.1 Tokens

❒ **Definition** – A *token* is an indivisible unit of text, such as a word, subword or character, and is part of a predefined vocabulary.

Remark: The unknown token `[UNK]` represents unknown pieces of text while the padding token `[PAD]` is used to fill empty positions to ensure consistent input sequence lengths.

❒ **Tokenizer** – A *tokenizer* T divides text into tokens of an arbitrary level of granularity.

this teddy bear is reaaaally cute → T → `this` `teddy bear` `is` `[UNK]` `cute` `[PAD]` ... `[PAD]`

Here are the main types of tokenizers:

Type	Pros	Cons	Illustration
Word	• Easy to interpret • Short sequence	• Large vocabulary size • Word variations not handled	`teddy` `bear`
Subword	• Word roots leveraged • Intuitive embeddings	• Increased sequence length • Tokenization more complex	`ted` `##dy` `bear`
Character Byte	• No out-of-vocabulary concerns • Small vocabulary size	• Much longer sequence length • Patterns hard to interpret because too low-level	`t` `e` `d` `d` `y` ... `b` `e` `a` `r`

Remark: Byte-Pair Encoding (BPE) and Unigram are commonly-used subword-level tokenizers.

1.2 Embeddings

❒ **Definition** – An *embedding* is a numerical representation of an element (e.g. token, sentence) and is characterized by a vector $x \in \mathbb{R}^n$.

❒ **Similarity** – The *cosine similarity* between two tokens t_1, t_2 is quantified by:

$$\text{similarity}(t_1, t_2) = \frac{t_1 \cdot t_2}{||t_1||\ ||t_2||} = \cos(\theta) \in [-1, 1]$$

The angle θ characterizes the similarity between the two tokens:

Remark: Approximate Nearest Neighbors (ANN) and Locality Sensitive Hashing (LSH) are methods that approximate the similarity operation efficiently over large databases.

2 Transformers

2.1 Attention

❒ **Formula** – Given a *query* q, we want to know which *key* k the query should pay "attention" to with respect to the associated *value* v.

Attention can be efficiently computed using matrices Q, K, V that contain queries q, keys k and values v respectively, along with the dimension d_k of keys:

$$\text{attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

❒ **MHA** – A *Multi-Head Attention* (MHA) layer performs attention computations across multiple heads, then projects the result in the output space.

It is composed of h attention heads as well as matrices W^Q, W^K, W^V that project the input to obtain queries Q, keys K and values V. The projection is done using matrix W^O.

Remark: Grouped-Query Attention (GQA) and Multi-Query Attention (MQA) are variations of MHA that reduce computational overhead by sharing keys and values across attention heads.

2.2 Architecture

❒ **Overview** – The *Transformer* is a landmark model relying on the self-attention mechanism and is composed of encoders and decoders. Encoders compute meaningful embeddings of the input that are then used by decoders to predict the next token in the sequence.

Stanford University — 1 — Fall 2026

VIP Cheatsheet

(also translated in 14 languages so far)

Link to PDF: https://github.com/afshinea/stanford-cme-295-transformers-large-language-models

Class communications

Canvas.

- Announcements
- Class discussions via Ed

Class communications

Canvas.

- Announcements
- Class discussions via Ed

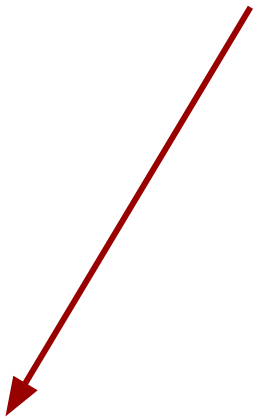
Any other general **inquiries** / **questions**, email us:

- cme295-aut2627-staff@lists.stanford.edu
- afshine@stanford.edu, shervine@stanford.edu

Example of a slide in this class

Explanation of a CME 295 concept

Source & suggested reading, if interested



Some paper

Disclaimer before starting: many abbreviations....

MHA LoRA RoPE AIME AttnRes PEFT GQA

Elo RLHF

QKNorm RM DPO LN GSPO

MoE

QLoRA GPQA SWA GRPO SFT

SwiGLU SWE-bench

PPO Pass@k TTFT MMLU TPOT

KL

MLA HLE PPL RLVR ALiBi MQA

...but don't worry!

MHA, GQA, MQA, MLA, SWA, AttnRes

Attention techniques

MoE, RoPE, ALiBi, LN, QKNorm, SwiGLU

Architecture optimizations

SFT, PEFT, LoRA, QLoRA, DPO

Model tuning

RLHF, RLVR, RM, PPO, GRPO, GSPO

Reinforcement Learning

AIME, HLE, MMLU, SWE-bench, GPQA

Benchmarks

PPL, LaaJ, KL, Pass@k, Elo, TTFT, TPOT

Metrics



CME 295 Transformers & Large Language Models

Introduction

Tokenization

Word representation

Self-attention mechanism

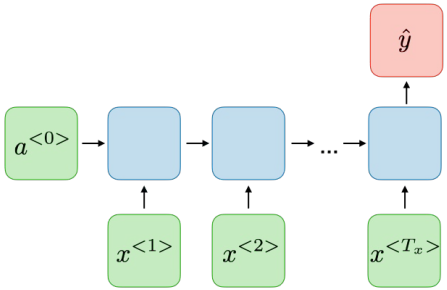
Transformer architecture

End-to-end example

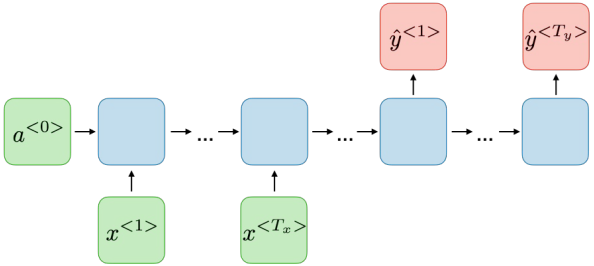
Discussion

Pre-LLM era: task-specific NLP models

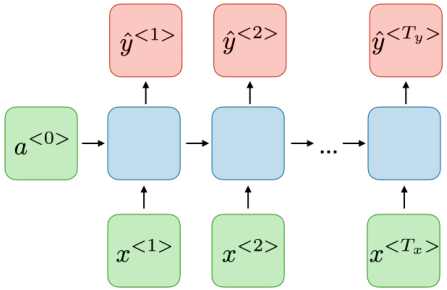
2010s



Sentiment extraction



Machine translation



Named entity recognition

Recurrent Neural Network cheatsheets, Amidi, 2018.

Attention is all you need paper

2010s

2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

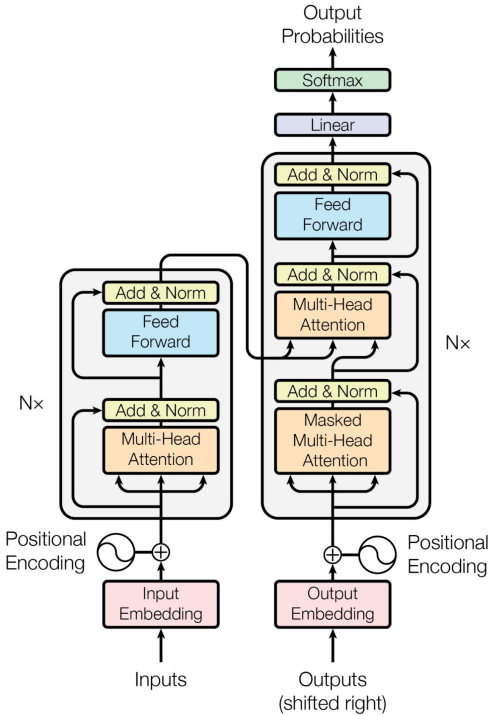
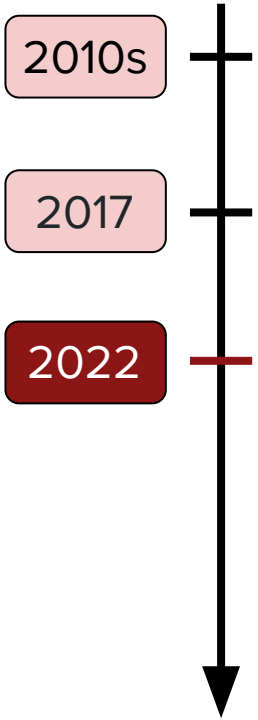


Figure 1: The Transformer - model architecture.

Launch of ChatGPT, "conversational" era



Sam Altman @sama

today we launched ChatGPT. try talking with it here:

chat.openai.com

11:38 AM · Nov 30, 2022

2.7K 16K 63K 22K

Sam Altman @sama · Nov 30, 2022

language interfaces are going to be a big deal, i think. talk to the computer (voice or text) and get what you want, for increasingly complex definitions of "want"!

this is an early demo of what's possible (still a lot of limitations--it's very much a research release).

29 143 2.5K

Sam Altman @sama · Nov 30, 2022

soon you will be able to have helpful assistants that talk to you, answer questions, and give advice. later you can have something that goes off and does tasks for you. eventually you can have something that goes off and discovers new knowledge for you.

106 530 5.2K

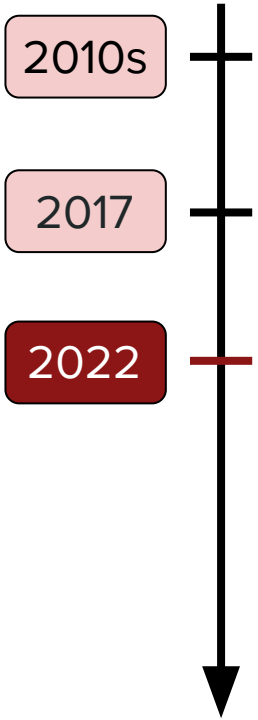
Sam Altman @sama · Nov 30, 2022

but this same interface works for all of that. this is something that scifi really god right; until we get neural interfaces, language interfaces are probably the next best thing.

37 113 1.8K

Tweet from Sam Altman announcing ChatGPT, 2022.

Launch of ChatGPT, "conversational" era



Sam Altman @sama

today we launched ChatGPT. try talking with it here:

chat.openai.com

11:38 AM · Nov 30, 2022

2.7K 16K 63K 22K

Sam Altman @sama · Nov 30, 2022

language interfaces are going to be a big deal, i think. talk to the computer (voice or text) and get what you want, for increasingly complex definitions of "want"!

this is an early demo of what's possible (still a lot of limitations--it's very much a research release).

29 143 2.5K

Sam Altman @sama · Nov 30, 2022

soon you will be able to have helpful assistants that talk to you, answer questions, and give advice. later you can have something that goes off and does tasks for you. eventually you can have something that goes off and discovers new knowledge for you.

106 530 5.2K

Sam Altman @sama · Nov 30, 2022

but this same interface works for all of that. this is something that scifi really god right; until we get neural interfaces, language interfaces are probably the next best thing.

37 113 1.8K

Sam Altman @sama · Nov 30, 2022

*got right! bad typo!!

50 36 1.2K

Tweet from Sam Altman announcing ChatGPT, 2022.

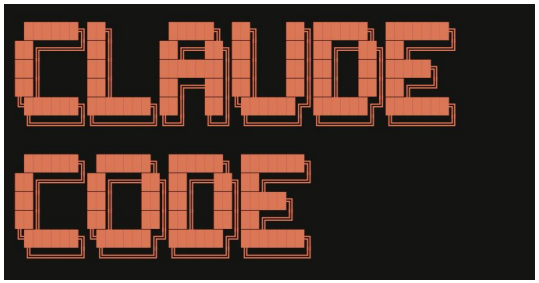
Agentic era!

2010s

2017

2022

Now



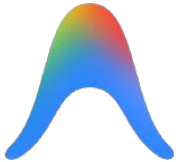
Anthropic's
Claude Code



Cursor



OpenAI's
Codex



Google's
Antigravity

Claude Code, *Cursor*, *Codex* and *Antigravity*.



CME 295 Transformers & Large Language Models

Introduction

Tokenization

Word representation

Self-attention mechanism

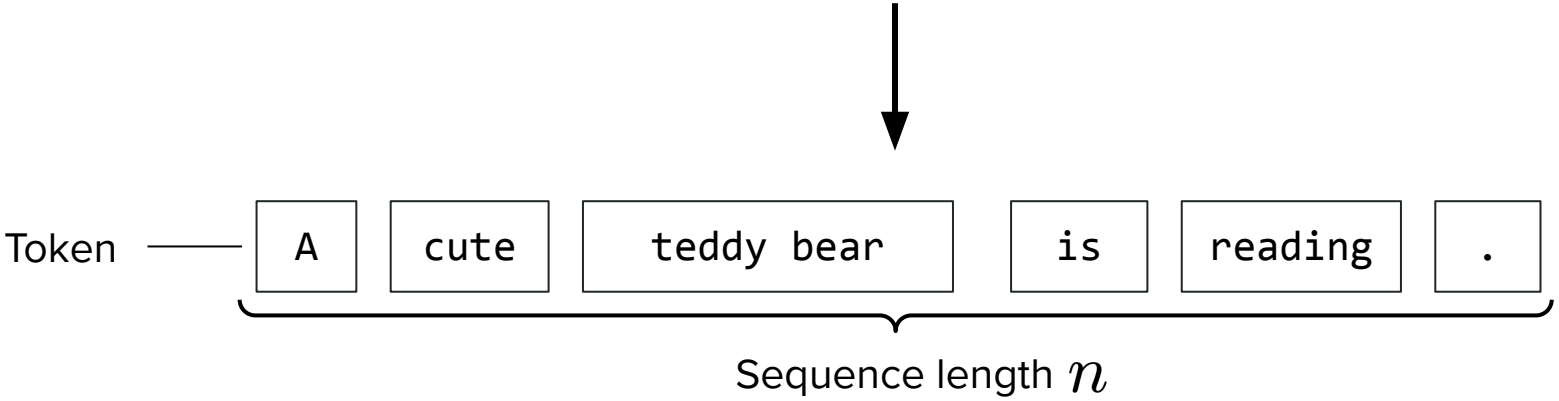
Transformer architecture

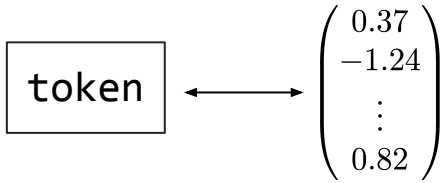
End-to-end example

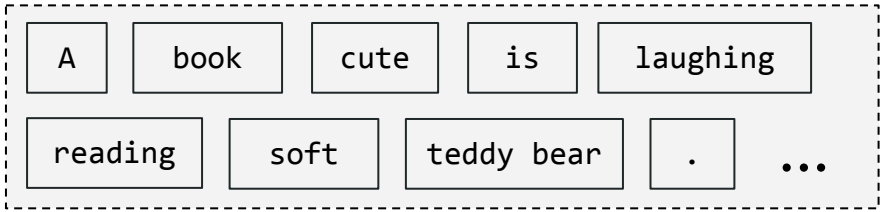
Discussion

Tokenization

A cute teddy bear is reading.







Vocabulary V

Tokenization: word-level

A cute teddy bear is reading.

Token = **word**



A	cute	teddy	bear	is	reading	.

🙂	😢
• Simple	• Vocabulary may be large
• Interpretable	• Does not leverage root of word
	• Risk of "out-of-vocabulary"

Tokenization: character-level

A cute teddy bear is reading.

Token = **character**



🙂	😢
• Robust to CASinG and mispelings	• Computations much slower
• Small vocabulary	• Embeddings not interpretable

Tokenization: subword-level

A cute teddy bear is reading.

Token = **subword**



A	cute	ted	##dy	bear	is	read	##ing	.

🙂

- Leverages roots of words
- Learned from the data

😢

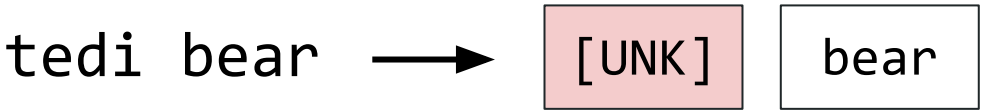
- Extra work
- Efficiency depends on training corpus

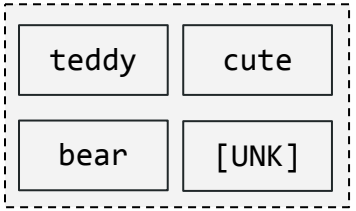
Suggested reading: Neural Machine Translation of Rare Words with Subword Units, Sennrich et al., 2015.

Special tokens

[UNK]

Unknown
("out of vocabulary")





Vocabulary V

Special tokens

[UNK]

Unknown
("out of vocabulary")

[BOS]

Beginning of
sequence

[EOS]

End of
sequence

What a cute teddy bear!



[BOS] What a cute teddy bear ! [EOS]

Special tokens

[UNK]

Unknown ("out of vocabulary")

[BOS]

Beginning of sequence

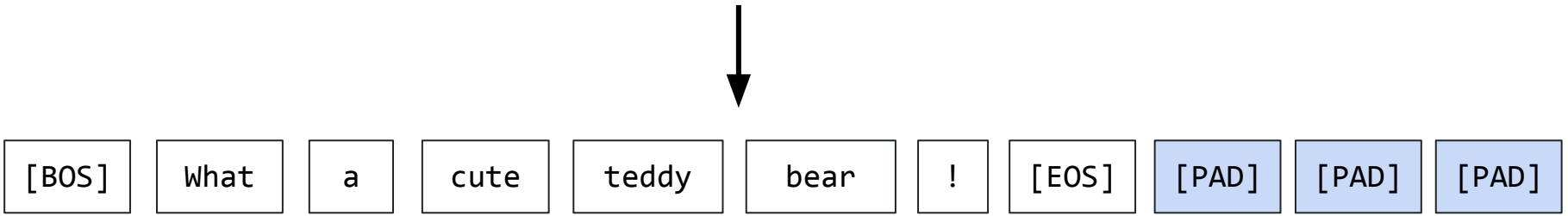
[EOS]

End of sequence

[PAD]

Padding

What a cute teddy bear!



Special tokens

[UNK]	[BOS]	[EOS]	[PAD]
Unknown ("out of vocabulary")	Beginning of sequence	End of sequence	Padding

Notes.

- Not exhaustive! (e.g. user/assistant for chat-based use cases)
- Many variations, different conventions (e.g. <|end_of_text|>)



CME 295 Transformers & Large Language Models

Introduction

Tokenization

Word representation

Self-attention mechanism

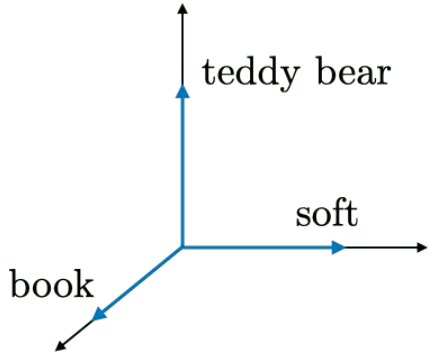
Transformer architecture

End-to-end example

Discussion

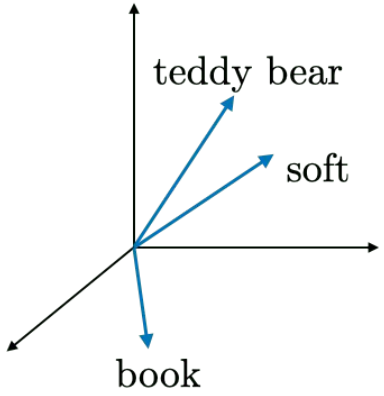
Token representations

Naive (one-hot) encoding



$$\text{soft} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \begin{aligned} \langle \text{teddy bear}, \ \text{book} \rangle &= 0 \\ \langle \text{teddy bear}, \ \text{soft} \rangle &= 0 \end{aligned}$$

Learned embedding



$$\text{soft} = \begin{pmatrix} 0.95 \\ 0.32 \\ 0.01 \end{pmatrix} \quad \begin{aligned} \langle \text{teddy bear}, \ \text{book} \rangle &\sim 0 \\ \langle \text{teddy bear}, \ \text{soft} \rangle &\sim 1 \end{aligned}$$

Figure adapted from VIP cheatsheets for Stanford's CS 230, Amidi, 2018.

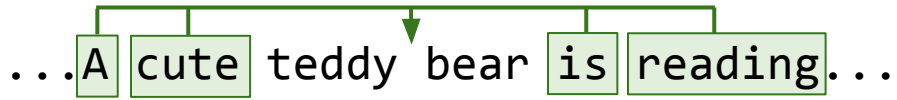
Word2vec

Overview

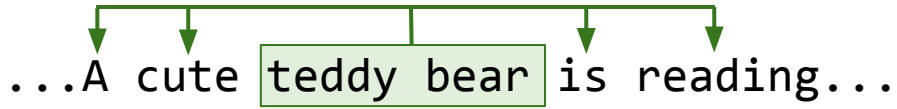
- Neural network with a **proxy task** over billions of words worth of text
- Learns an embedding layer

Proxy tasks

- CBOW (continuous bag of words)



- Skip-gram



Efficient Estimation of Word Representations in Vector Space, Mikolov et al., 2013.

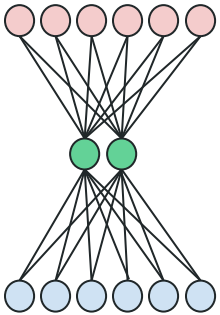
Word2vec

Architecture

output

hidden

input



size V

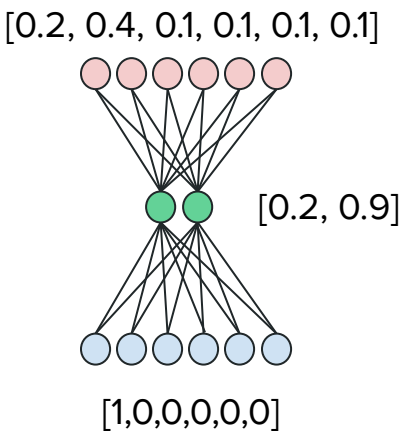
size d

size V

Word2vec

Example with predicting next word

A [cute] teddy bear is reading

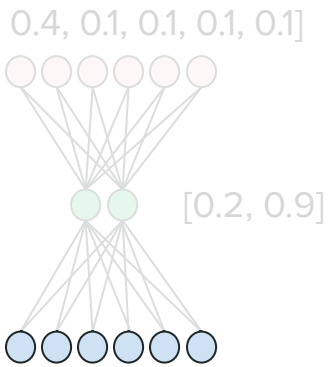


[A] cute teddy bear is reading

Word2vec

Example with predicting next word

A cute teddy bear is reading



One-hot encoding

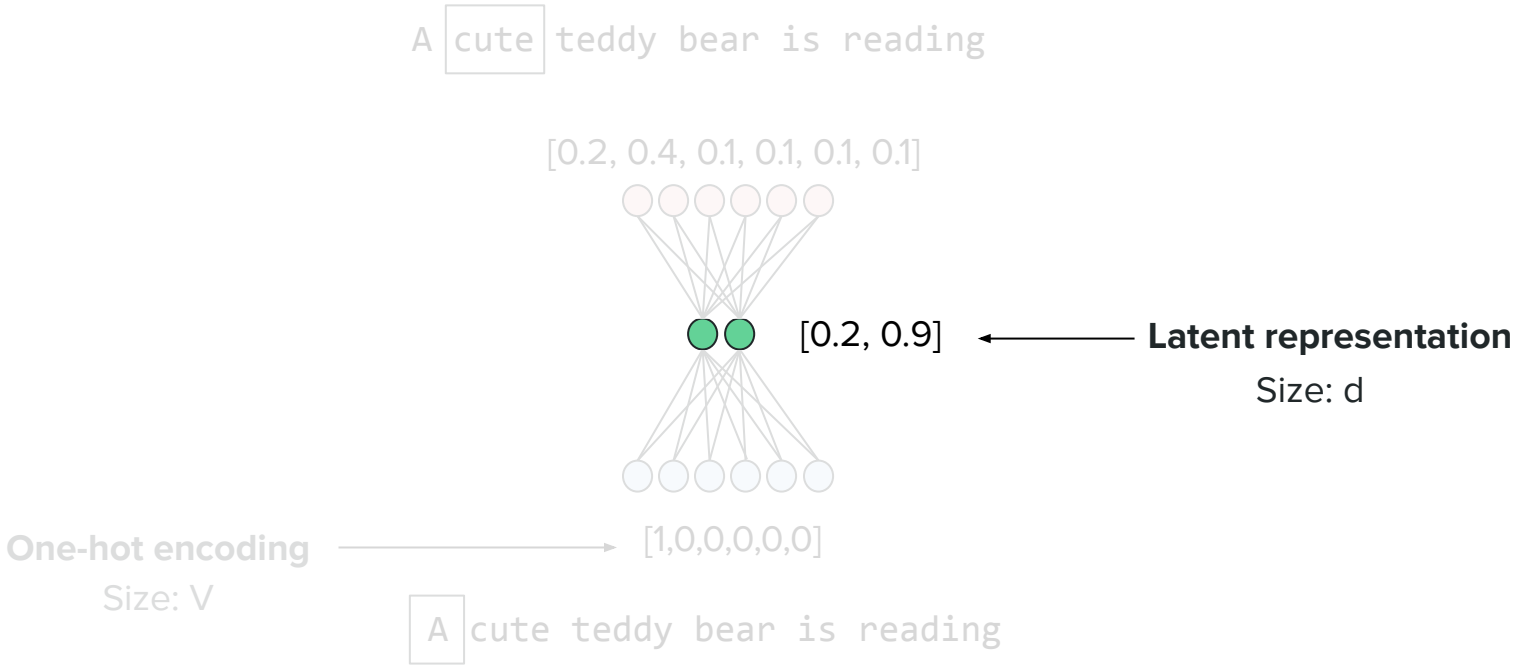
Size: V

[1,0,0,0,0,0]

A cute teddy bear is reading

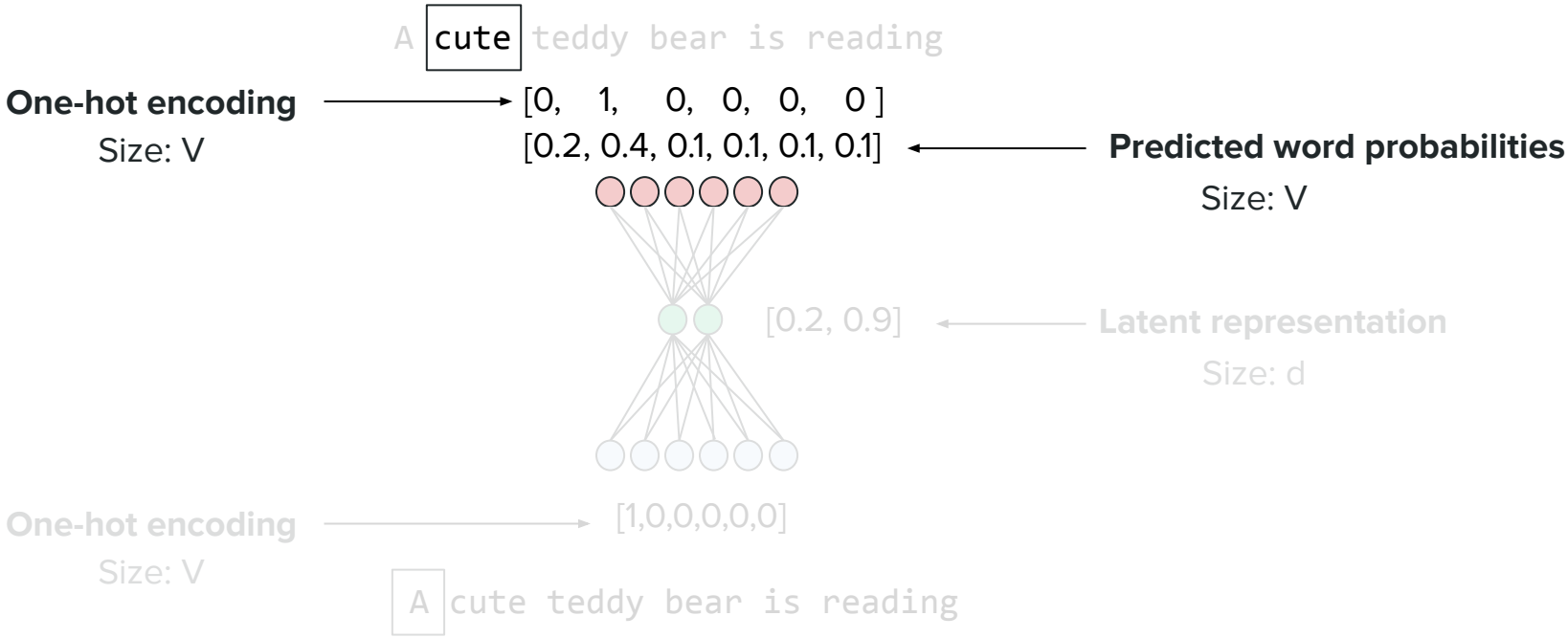
Word2vec

Example with predicting next word



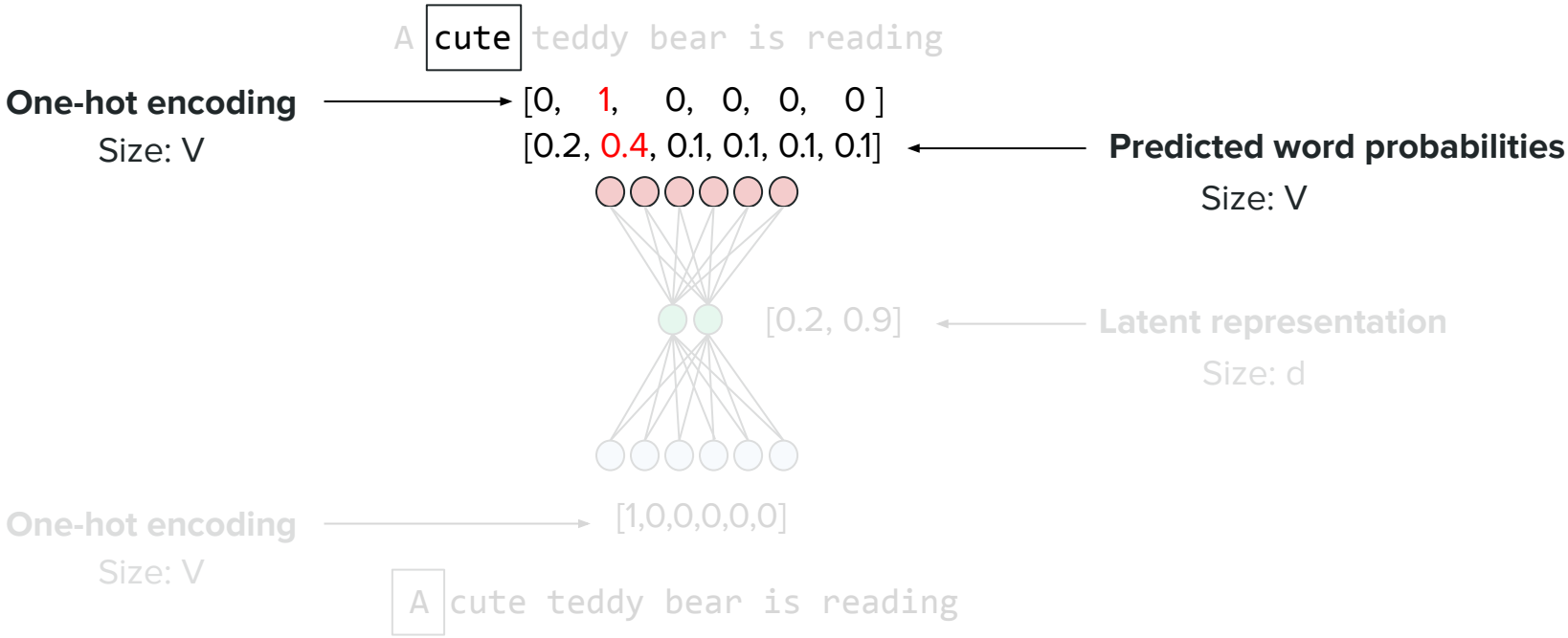
Word2vec

Example with predicting next word



Word2vec

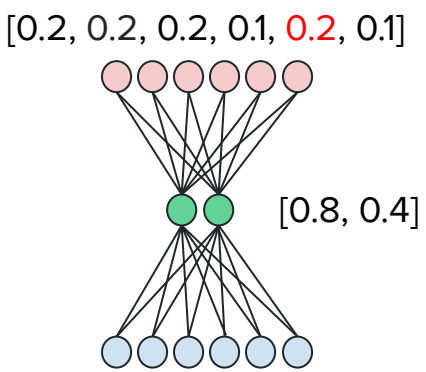
Example with predicting next word



Word2vec

Example with predicting next word

A cute **teddy bear** is reading

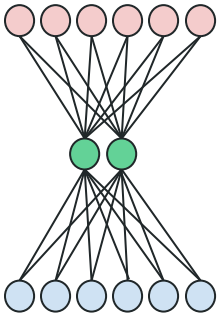


A **cute** teddy bear is reading

Word2vec

Example with predicting next word

A cute teddy bear **is** reading

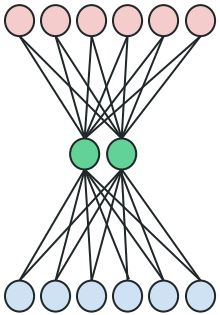


A cute **teddy bear** is reading

Word2vec

Example with predicting next word

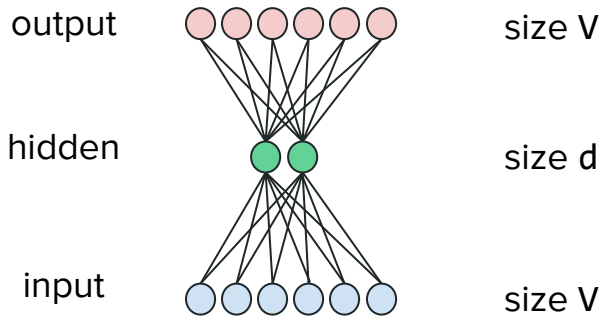
A cute teddy bear is **reading**



A cute teddy bear **is** reading

Word2vec

A **cute** teddy bear is reading



A cute teddy bear is reading

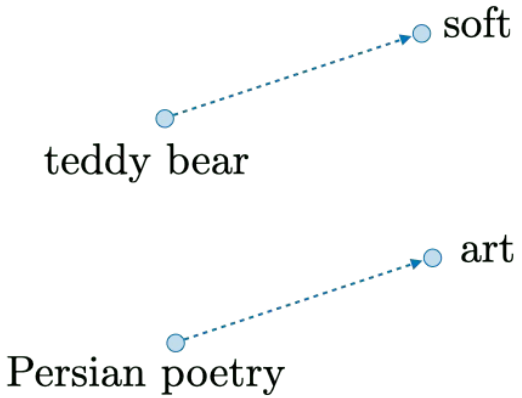


Figure adapted from VIP cheatsheets for Stanford's CS 230, Amidi, 2018.

Recurrent Neural Networks (RNNs)

Overview

- First introduced in the 80s
- Class of neural networks where connections form a temporal sequence

General form

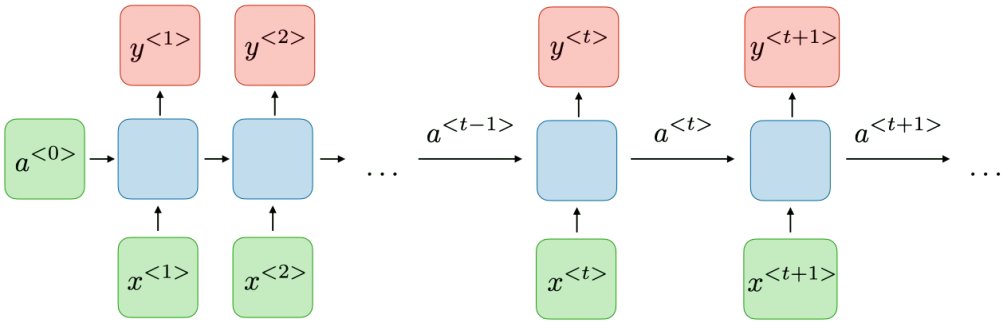
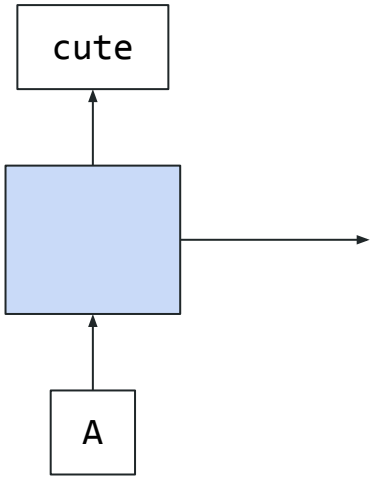
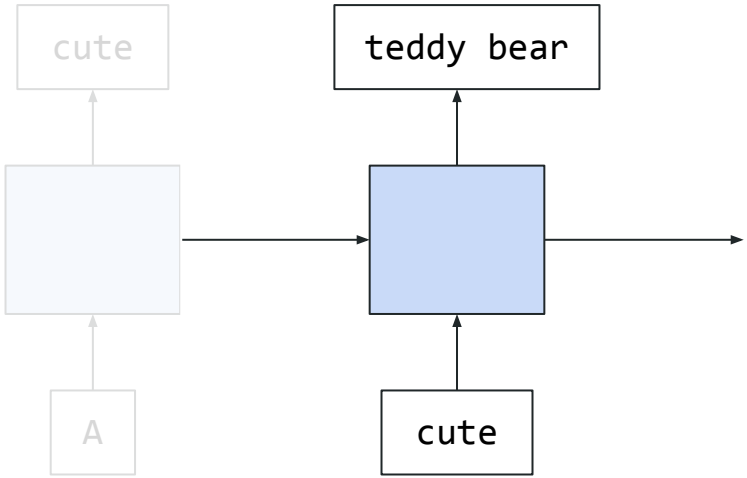


Figure adapted from VIP cheatsheets for Stanford's CS 230, Amidi, 2018.

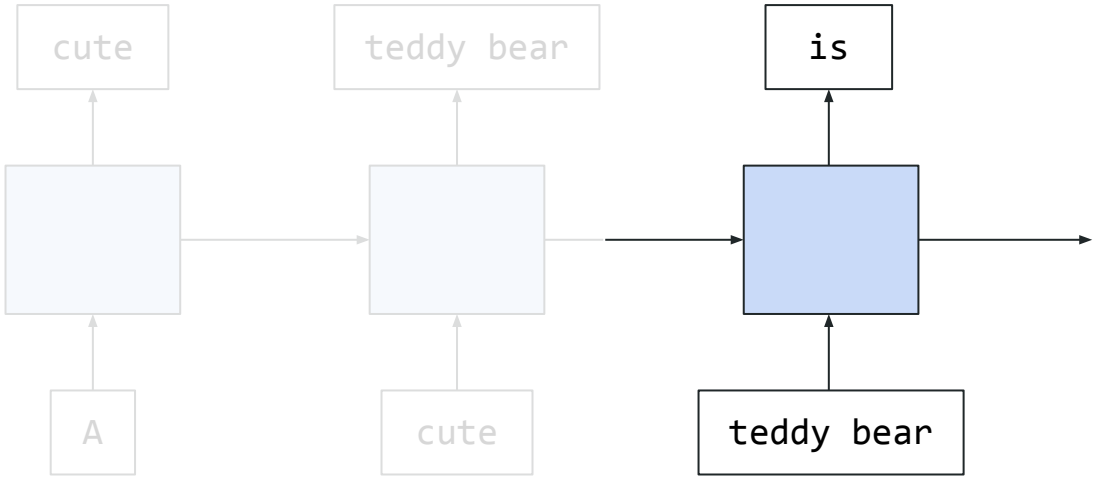
Recurrent Neural Networks (RNNs)



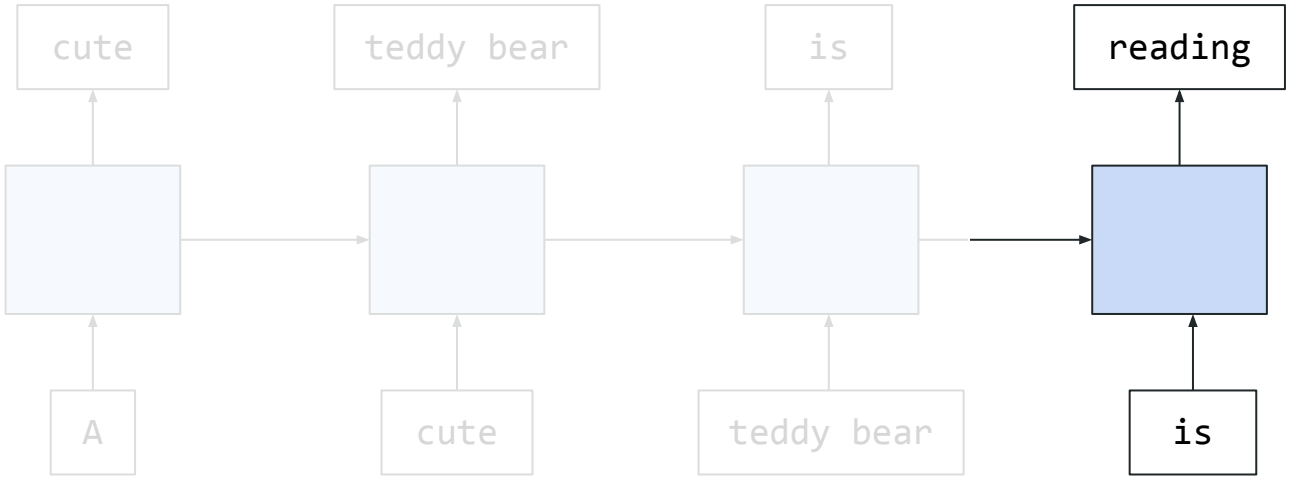
Recurrent Neural Networks (RNNs)



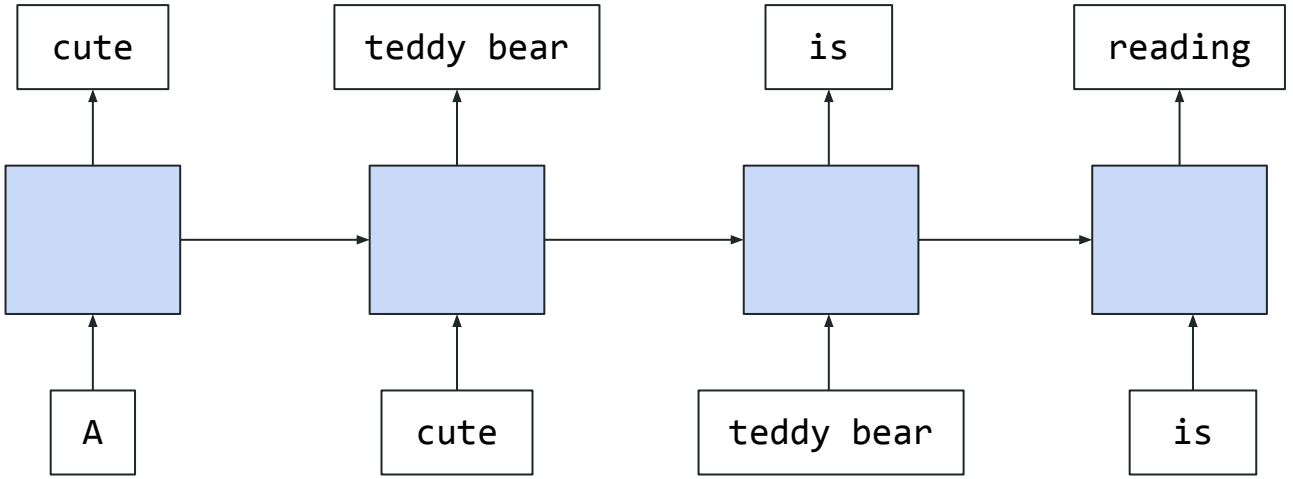
Recurrent Neural Networks (RNNs)



Recurrent Neural Networks (RNNs)

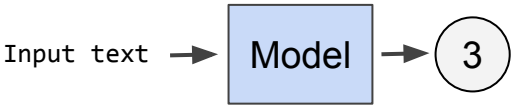


Recurrent Neural Networks (RNNs)

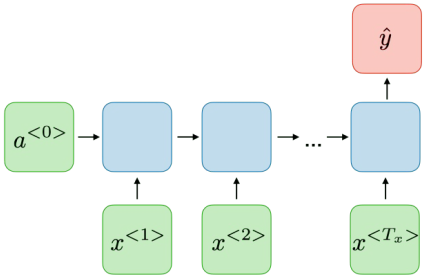


Recurrent Neural Networks (RNNs)

Classification

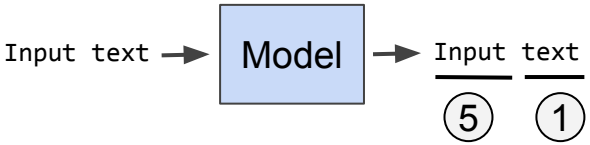


Sentiment

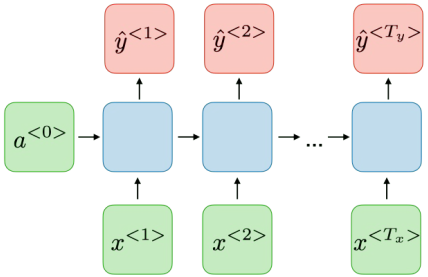


Opinion

"Multi"-classification

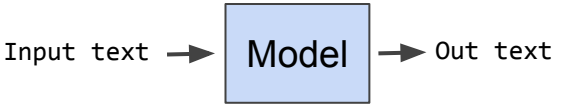


Tags

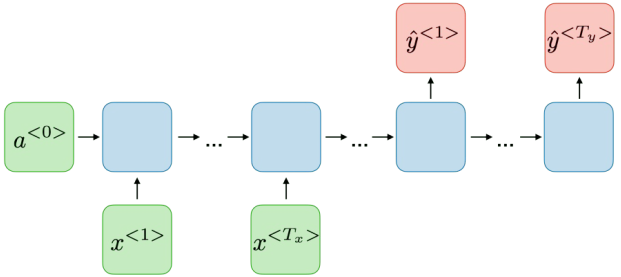


Text

Generation



Translation



Source

Figure adapted from VIP cheatsheets for Stanford's CS 230, Amidi, 2018.

Long Short-Term Memory (LSTM)

Overview

- Introduced in the Long short-term memory paper (1997)
- Uses a more structured approach in the cell's hidden state

General form

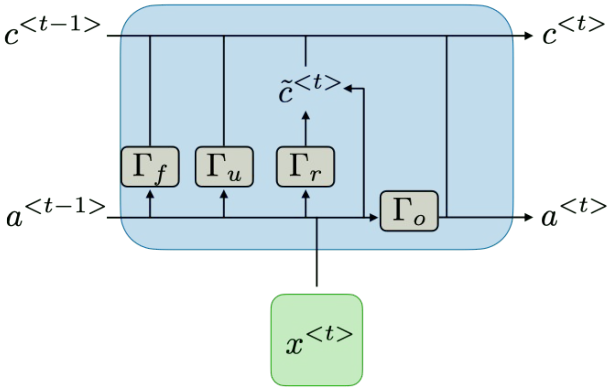


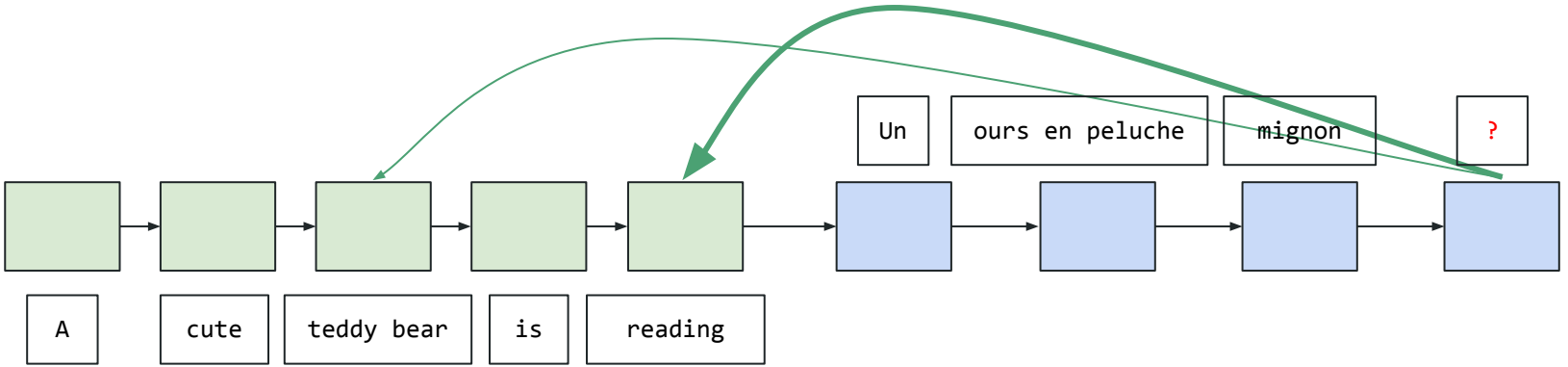
Figure adapted from VIP cheatsheets for Stanford's CS 230, Amidi, 2018.

Summary of main methods (non-exhaustive list)

Method	Pros	Cons
Word2vec e.g. CBOW, Skip-gram	• Very simple, yet powerful • Intuitive embeddings	• Word order does not count • Embeddings not context aware
Recurrent Neural Networks e.g. traditional RNN, LSTM	• Word order matters • State-of-the-art results at the time	• Vanishing gradient problem • Slow computations

History of attention

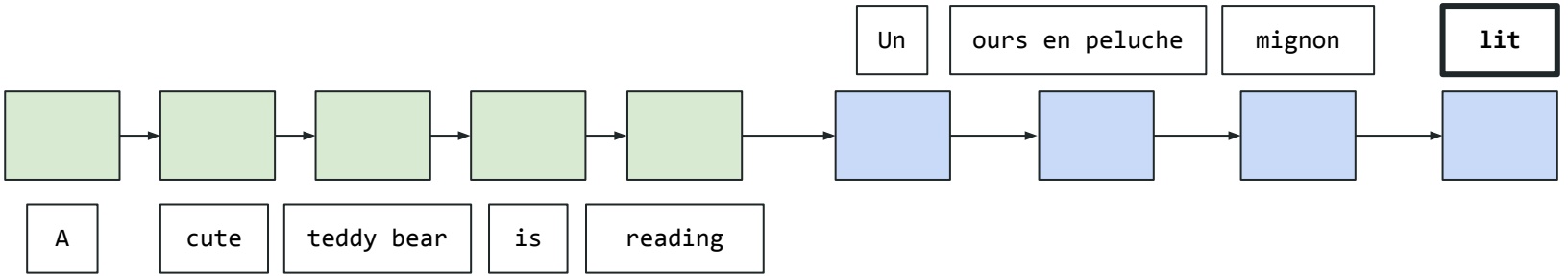
- Introduced in 2014
- Translation tasks had a real issue with long-term dependencies
- Seq2seq unable to "remember" what input sentence was saying



Neural Machine Translation by Jointly Learning to Align and Translate, Bahdanau et al., 2014.

History of attention

- Introduced in 2014
- Translation tasks had a real issue with long-term dependencies
- Seq2seq unable to "remember" what input sentence was saying



Neural Machine Translation by Jointly Learning to Align and Translate, Bahdanau et al., 2014.



CME 295 Transformers & Large Language Models

Introduction

Tokenization

Word representation

Self-attention mechanism

Transformer architecture

End-to-end example

Discussion

Overview of the Transformer

- Introduced in the Attention is All You Need paper, published in 2017
- Relies on the **self-attention** mechanism
- State-of-the-art results on machine translation tasks

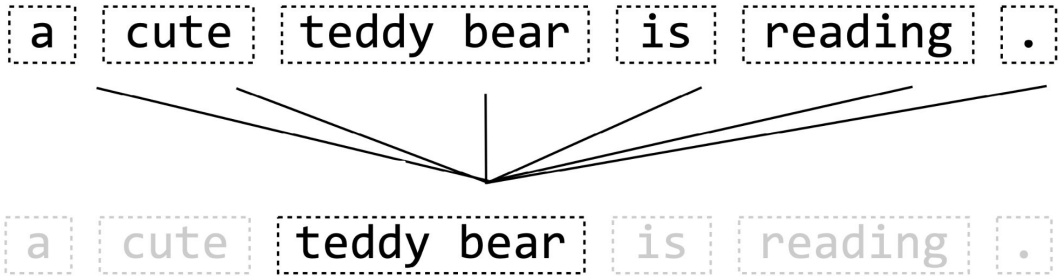


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Attention mechanism

Concept of **Q**uery, **K**ey, **V**alue

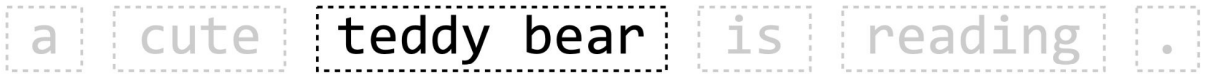


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Attention mechanism

Concept of **Q**uery, **K**ey, **V**alue

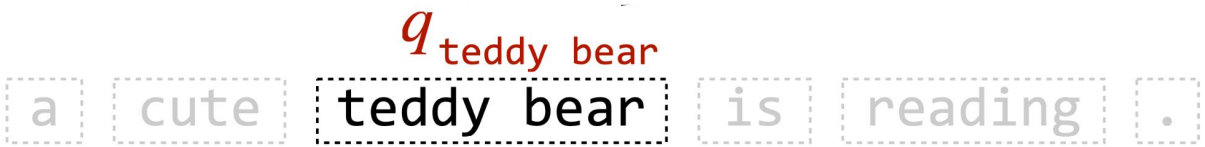


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Attention mechanism

Concept of **Q**uery, **K**ey, **V**alue

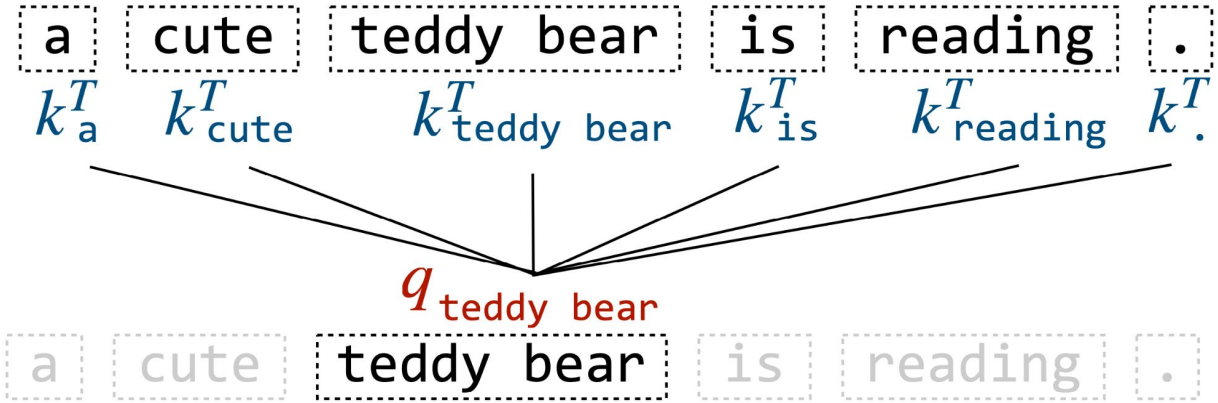


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Attention mechanism

Concept of **Q**uery, **K**ey, **V**alue

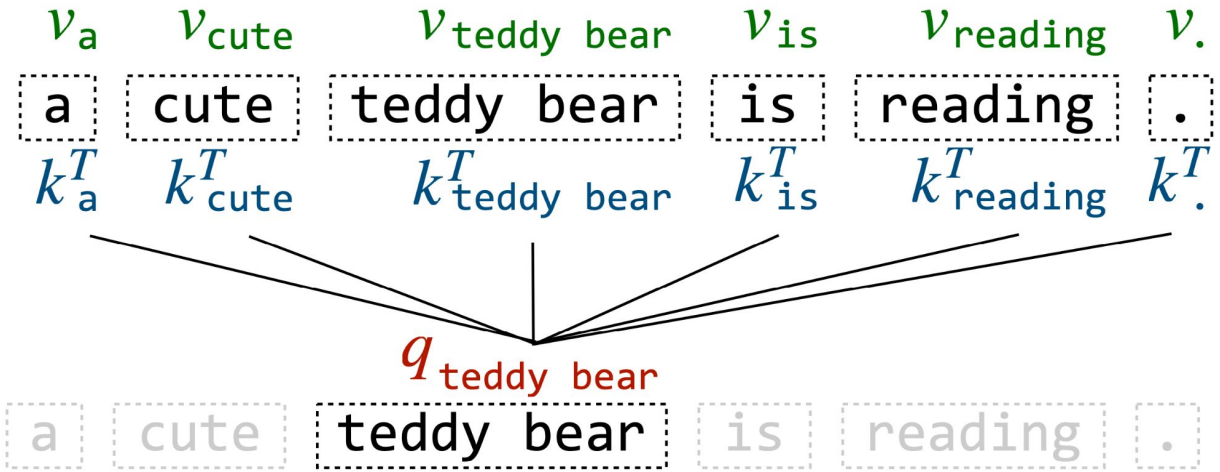


Figure from Super Study Guide: Transformers & Large Language Models, Amidi, 2024.

Attention mechanism

Efficient computations with matrices:

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

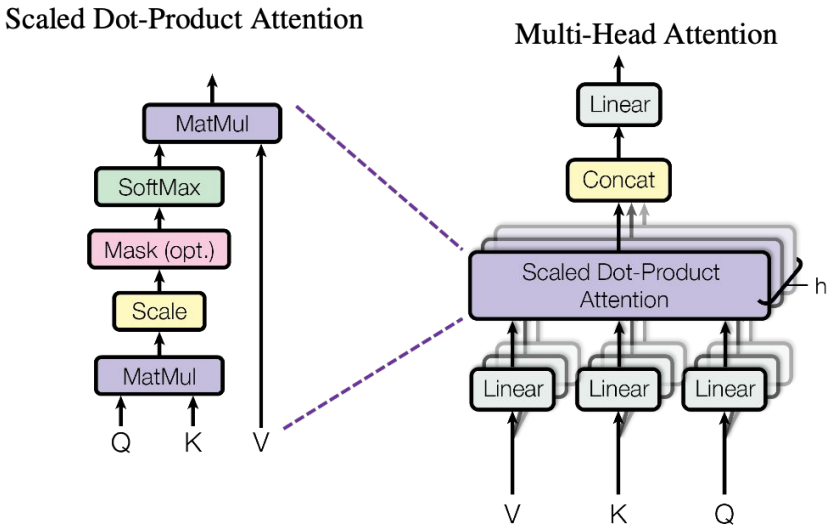


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.



CME 295 Transformers & Large Language Models

Introduction

Tokenization

Word representation

Self-attention mechanism

Transformer architecture

End-to-end example

Discussion

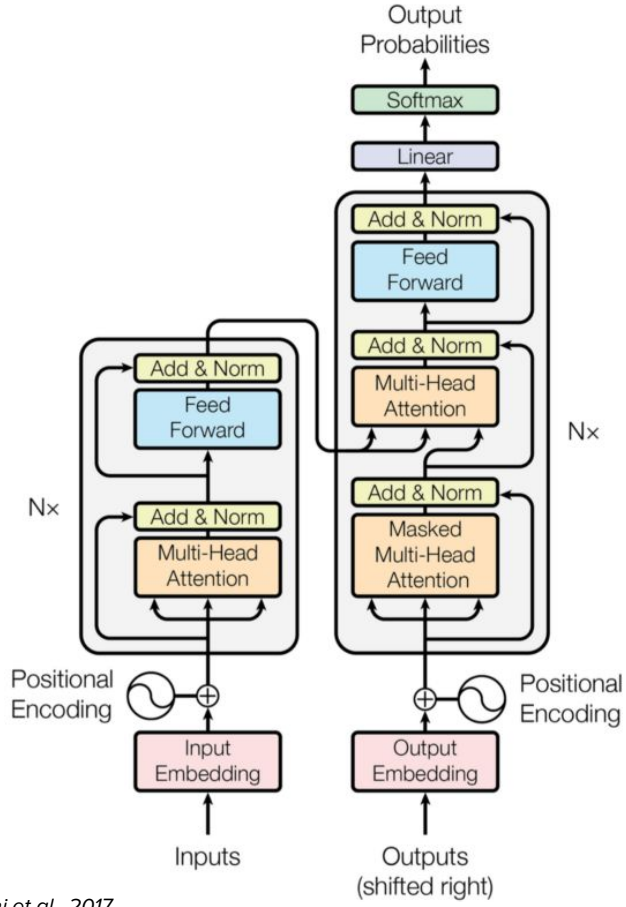
Transformer architecture

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.



Un ours en peluche mignon lit.

A cute teddy bear is reading.

Attention Is All You Need, Vaswani et al., 2017.

Transformer architecture

"Compute meaningful embeddings"

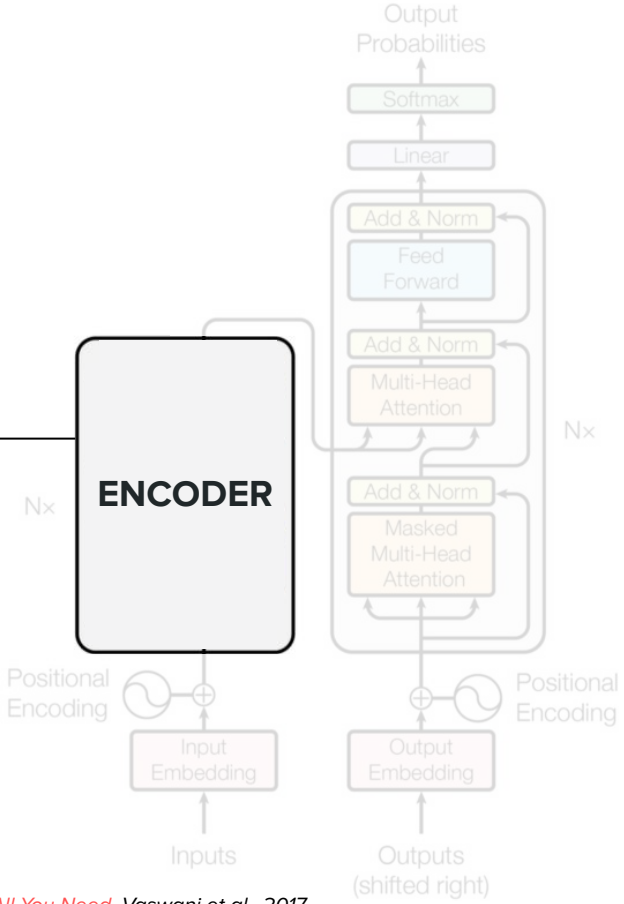


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Transformer architecture

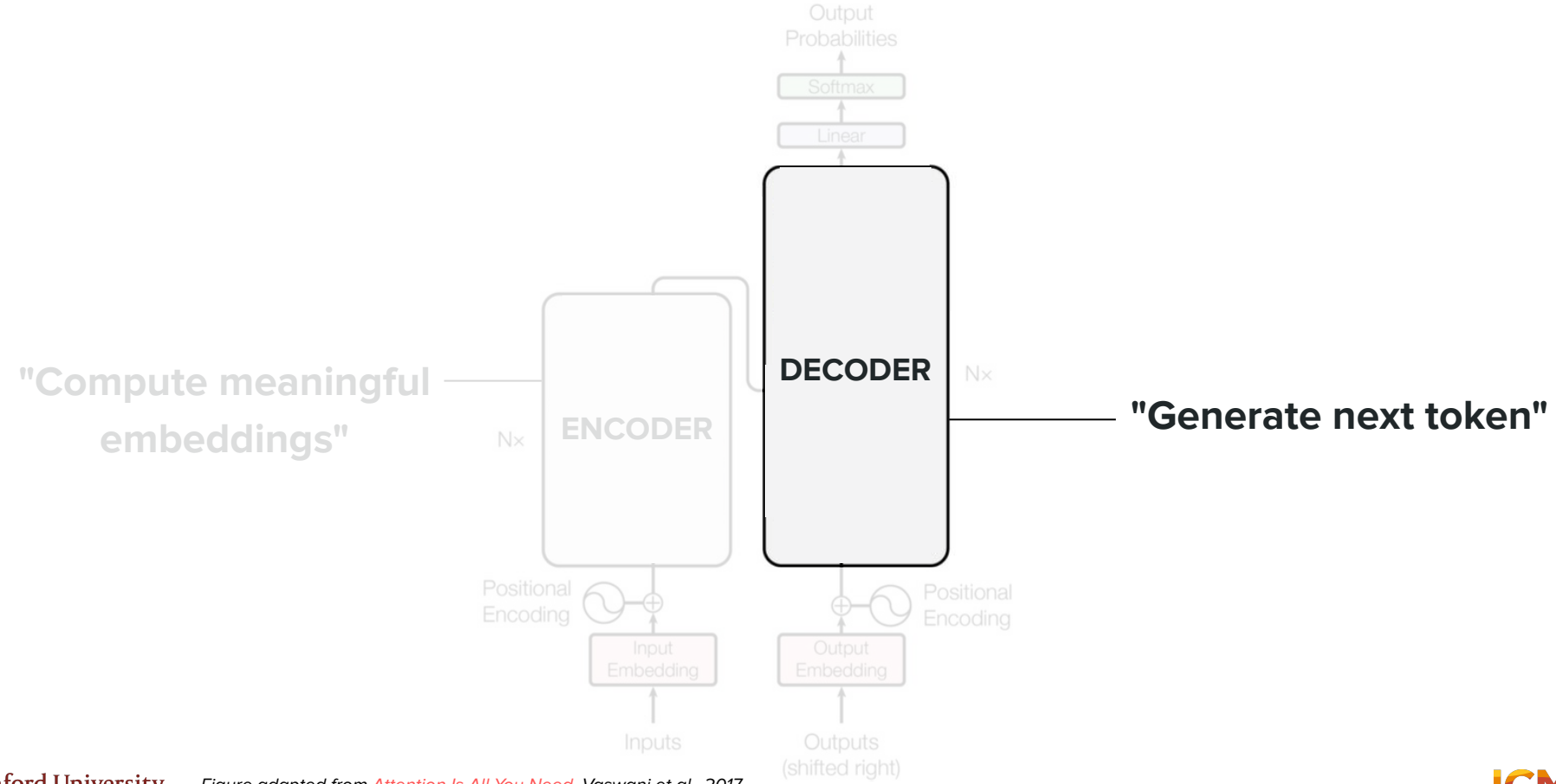
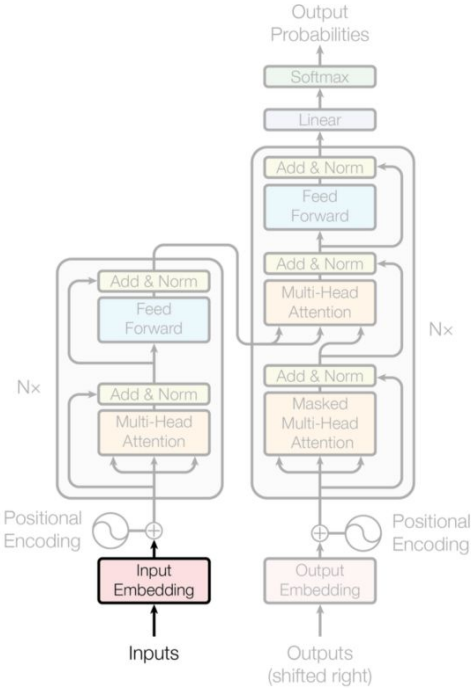


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Encoder input...



Encoder input

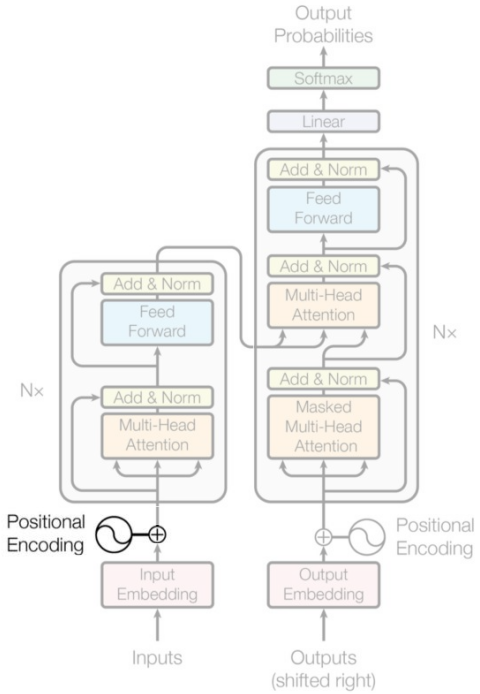
- Text is "tokenized"
- Each token has a learned embedding

Parameters

- V: vocabulary size
- `d_model`: embedding dimensions

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

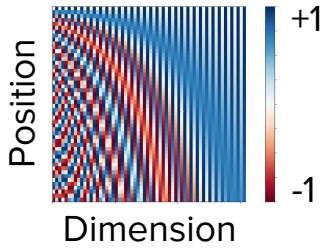
...with a trick!

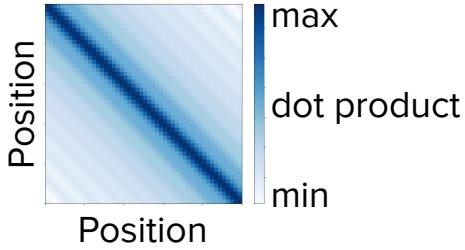


Positional encoding

Idea:

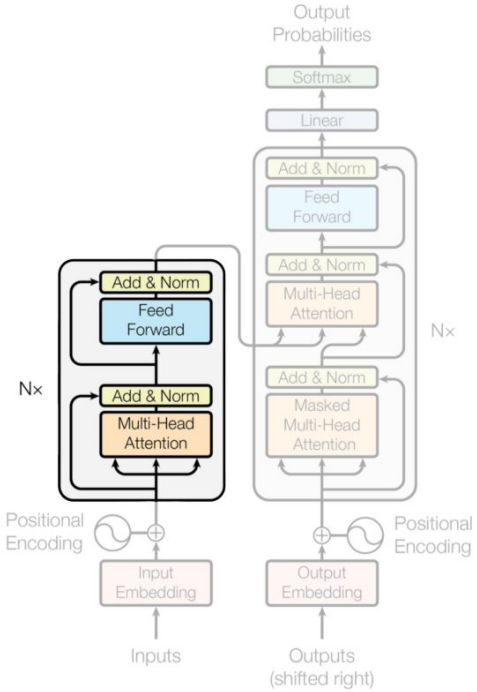
- Add **position information** to inputs
- Can be either learned or hardcoded





Left figure adapted from Attention Is All You Need, Vaswani et al., 2017.
Right figures adapted from Transformer Architecture: The Positional Encoding, Kazemnejad, 2019.

Encoder



Encoder components

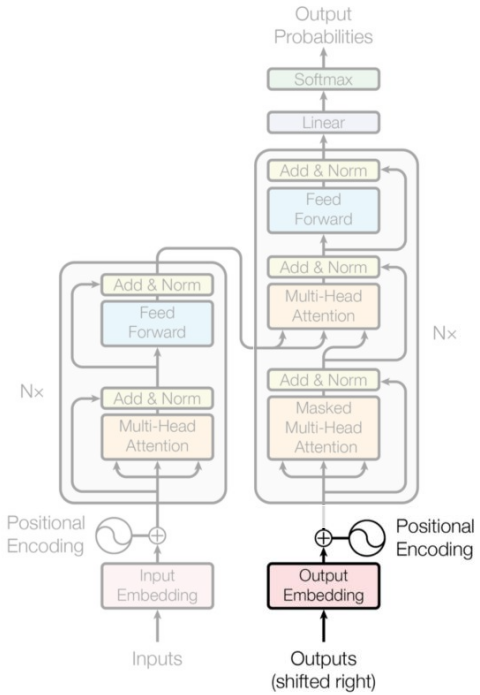
- Encoder-Encoder attention / self-attention
- Feed Forward Neural Network
- Normalization layer

Parameters

- `N`: layers stacked
- `h`: number of attention heads
- `d_FF`, `d_key`, `d_value`: sub-layer dimension
- `d_model`: embedding dimensions

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Decoder input, with position information



Decoder input

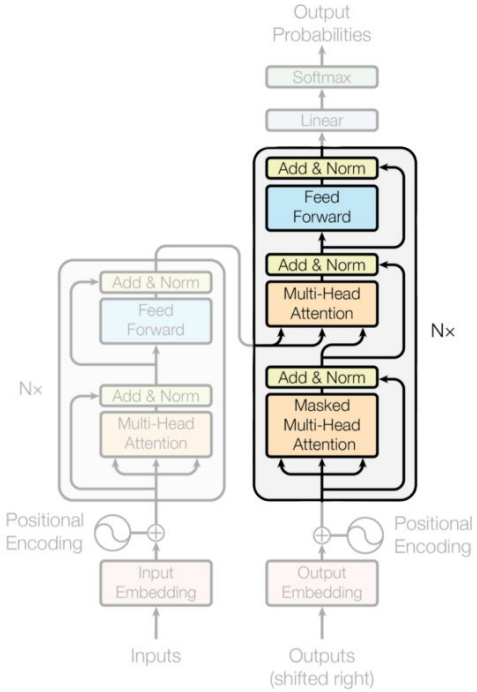
- Learned embeddings for output tokens
- In practice, will start with `[BOS]` during translation

Parameters

- V: vocabulary size
- `d_model`: embedding dimensions

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Decoder



Decoder components

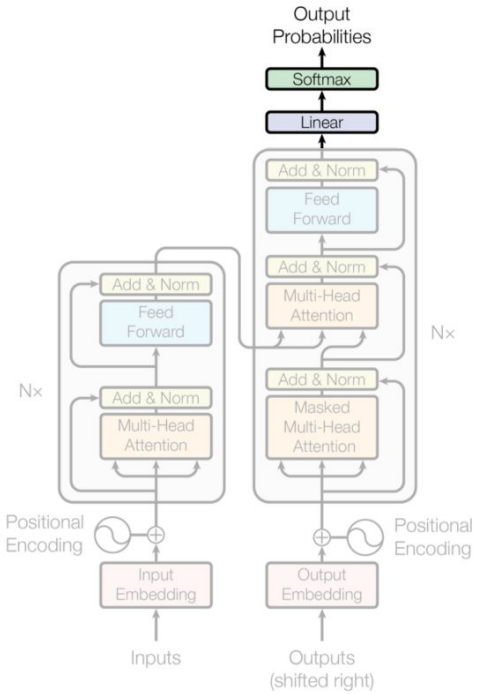
- Decoder-Decoder attention / self-attention
- Encoder-Decoder attention
- Feed Forward Neural Network
- Normalization layer

Parameters

- N: layers stacked
- h: number of attention heads
- d_FF, d_key, d_value: sub-layer dimension
- d_model: embedding dimensions

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Output



Decoder outputs

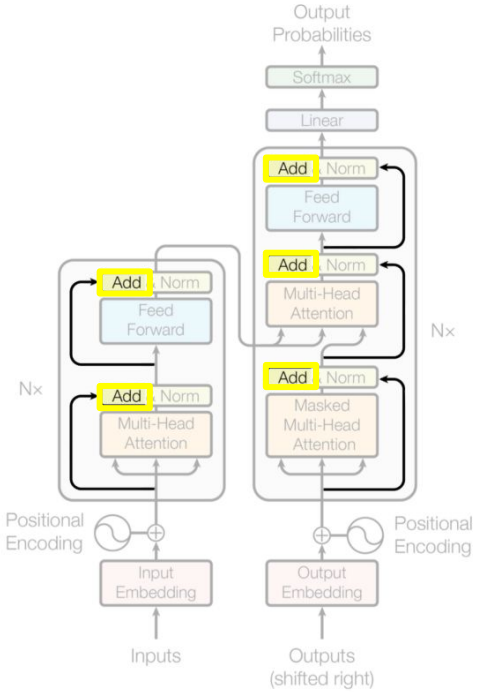
- Linear projection
- Classification problem that outputs probability of belonging to a class, where class = word

Parameters

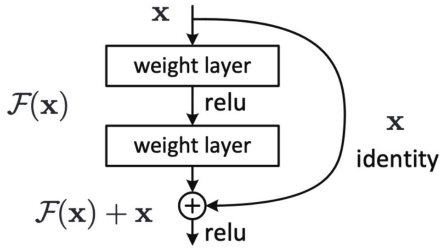
- V: vocabulary size
- `d_model`: embedding dimensions

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Computational trick: residual connections



Residual connections



Idea:

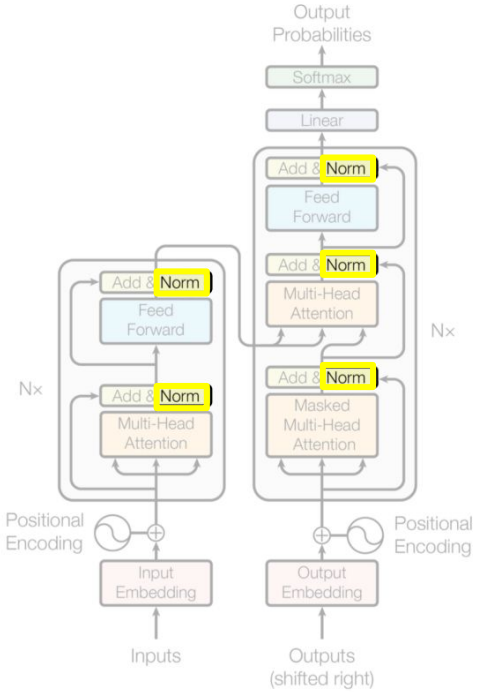
- Taken from the ResNet paper
- Element-wise addition of input and output features at different stages of the network

Benefits:

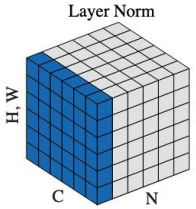
- Diversifies features at each level of the network
- Helps with gradient propagation

Left figure adapted from Attention Is All You Need, Vaswani et al., 2017. Right figure from Deep Residual Learning for Image Recognition, He et al., 2015.

Computational trick: layer normalization



Layer normalization



Idea:

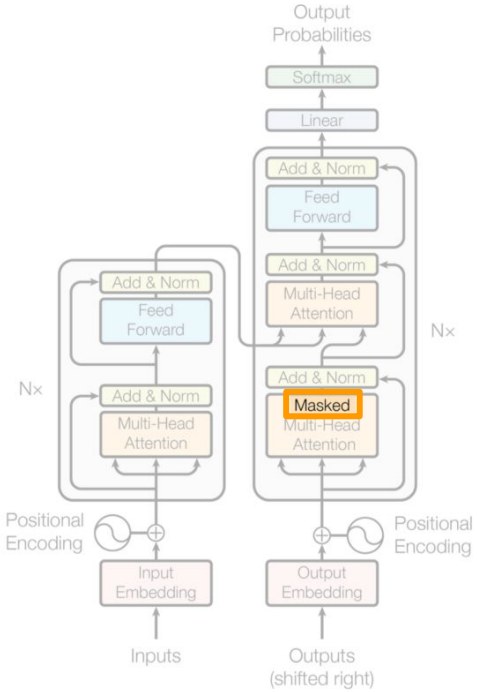
- Normalize activation outputs over neurons of a hidden layer
- Keeps scale of activations consistent across layers

Benefits:

- Helps with gradient propagation
- Faster, more stable convergence in practice

Left figure adapted from Attention Is All You Need, Vaswani et al., 2017. Right figure adapted from Group normalization, Wu et al., 2018.

Computational trick: masking



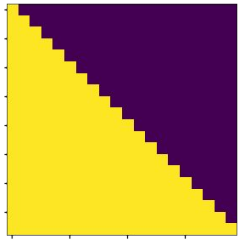
Masking

"Masked" also known as "causal" self-attention

Idea:

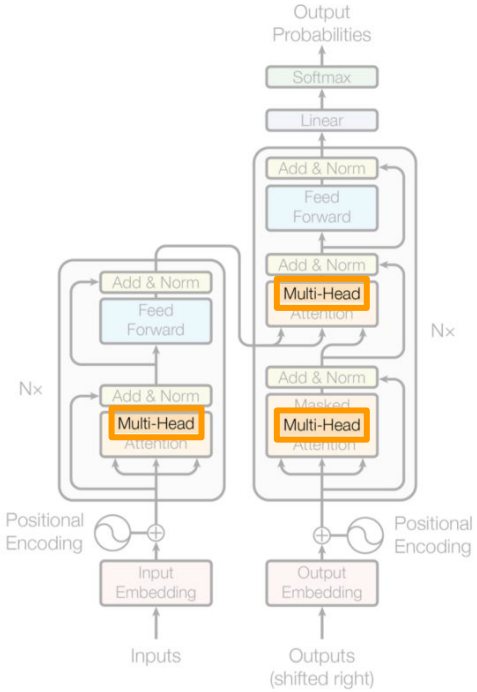
- Future tokens to be decoded need to be hidden at training time
- Create matrix of masked tokens at each decoding step

Benefit: All operations are vectorized



Left figure adapted from Attention Is All You Need, Vaswani et al., 2017. Right figure from The Annotated Transformer, Harvard NLP, 2018.

Computational trick: multi-head attention

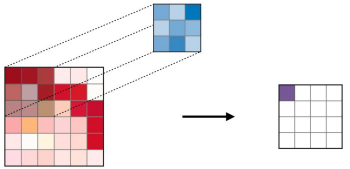


Multi-head attention

Idea: Run **multiple** self-attention layers in parallel

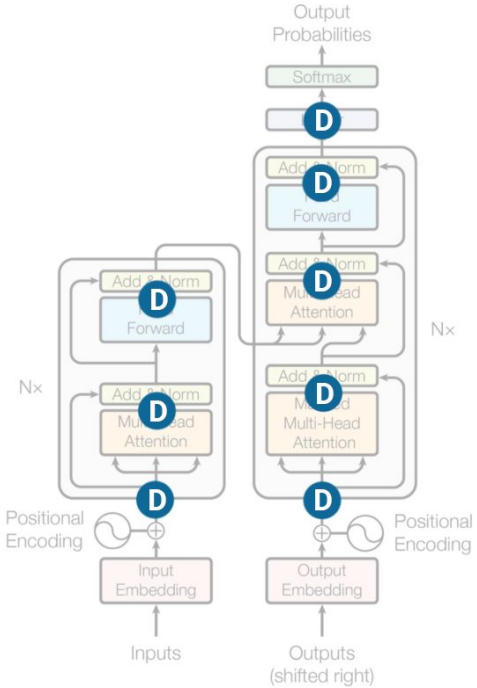
Benefits:

- Enables the model to capture different attention features in parallel
- Comparison: **multiple** filters of a convolutional layer in computer vision



Left figure adapted from Attention Is All You Need, Vaswani et al., 2017. Right figure from VIP cheatsheets for Stanford's CS 230, Amidi, 2018.

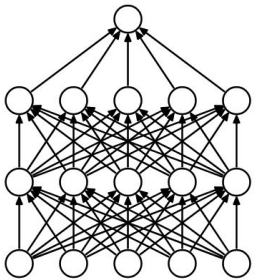
Computational tricks



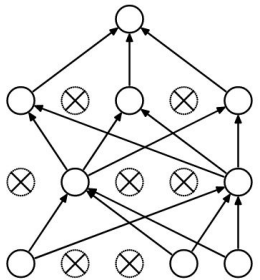
Dropout

Idea: Randomly drop neural net connections/units with a small probability

Better **generalization**



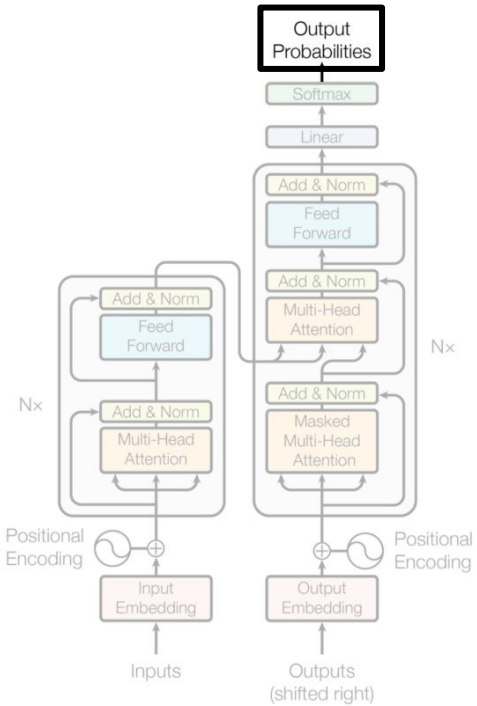
(a) Standard Neural Net



(b) After applying dropout.

Left figure adapted from Attention Is All You Need, Vaswani et al., 2017.
Right figure from Dropout: A Simple Way to Prevent Neural Networks from Overfitting, Srivastava et al., 2014.

Computational tricks



Label smoothing

Idea:

- 2015 vision paper: overconfidence is bad
- Introduce **noise** in true labels

$$q(k|x) = \delta_{k,y} \longrightarrow q'(k|x) = (1-\epsilon)\delta_{k,y} + \epsilon u(k)$$

Benefits:

- General technique that prevents overfitting
- Improves accuracy and BLEU score

Figure adapted from Attention Is All You Need, Vaswani et al., 2017.



CME 295 Transformers & Large Language Models

Introduction

Tokenization

Word representation

Self-attention mechanism

Transformer architecture

End-to-end example

Discussion

Stitching all the pieces together with an example

```
A cute teddy bear is reading.
```

Stitching all the pieces together with an example



Stitching all the pieces together with an example



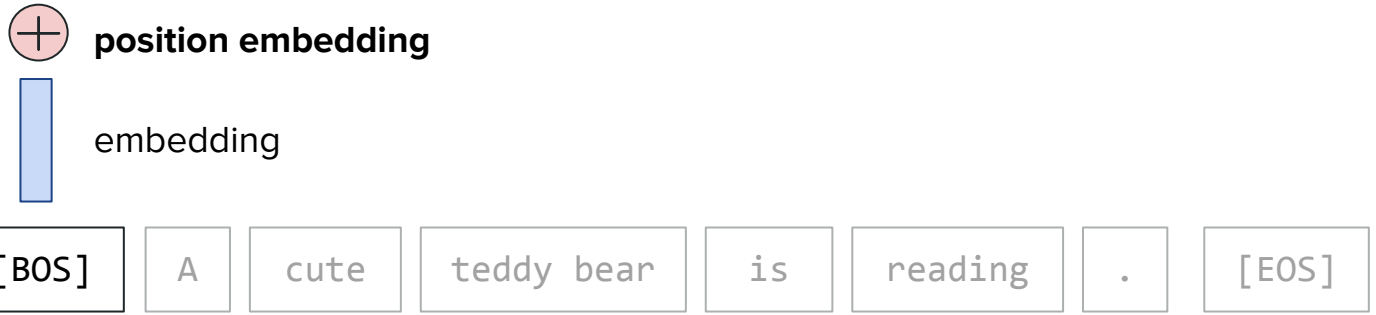
Stitching all the pieces together with an example



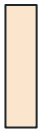
Stitching all the pieces together with an example



Stitching all the pieces together with an example



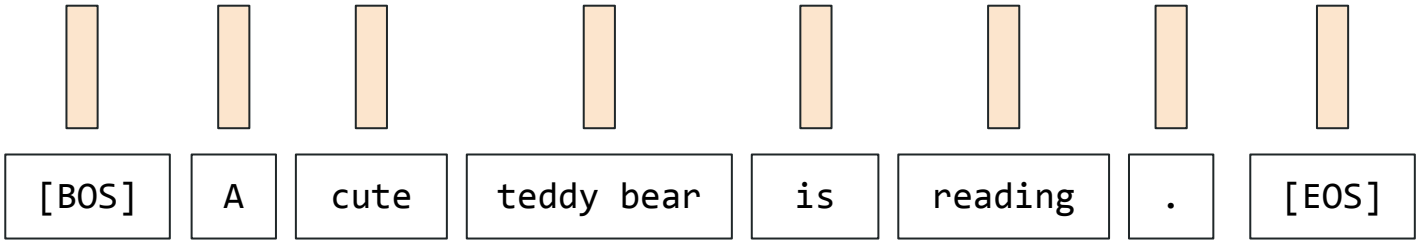
Stitching all the pieces together with an example



position-aware embedding

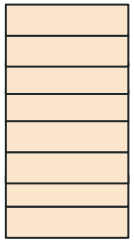
[BOS]	A	cute	teddy bear	is	reading	.	[EOS]

Stitching all the pieces together with an example



position-aware embeddings

Stitching all the pieces together with an example



position-aware embeddings matrix

[BOS]	A	cute	teddy bear	is	reading	.	[EOS]

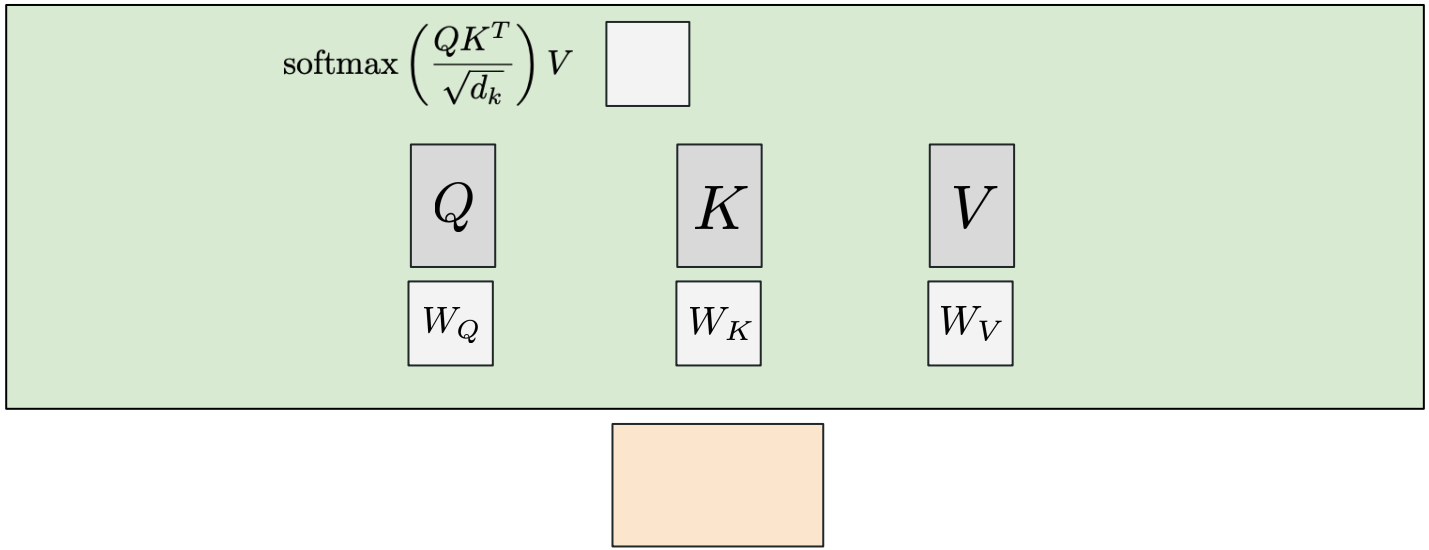
Stitching all the pieces together with an example



position-aware embeddings matrix

[BOS]	A	cute	teddy bear	is	reading	.	[EOS]

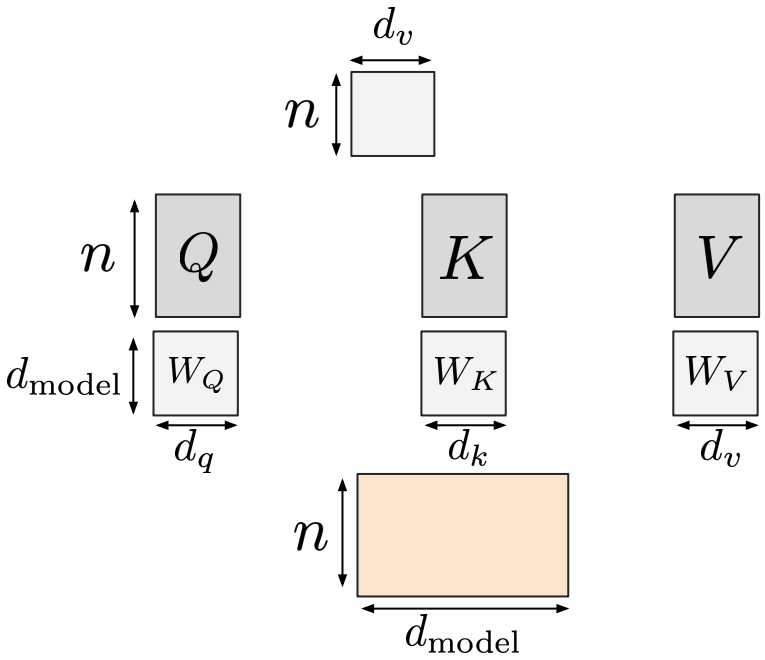
Stitching all the pieces together with an example



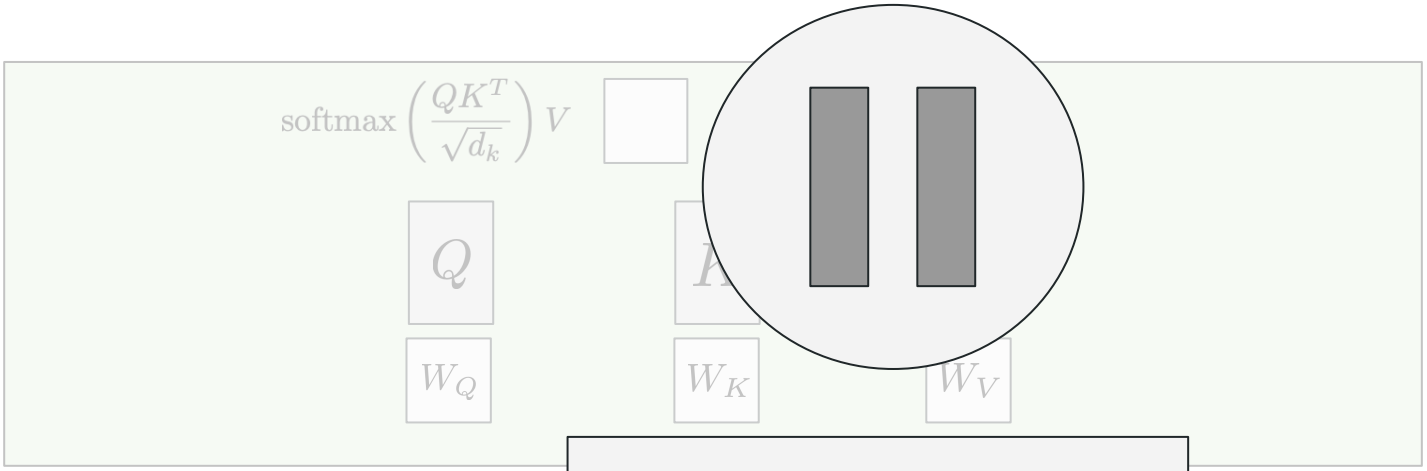
encoder

position-aware embeddings matrix

Stitching all the pieces together with an example



Stitching all the pieces together with an example



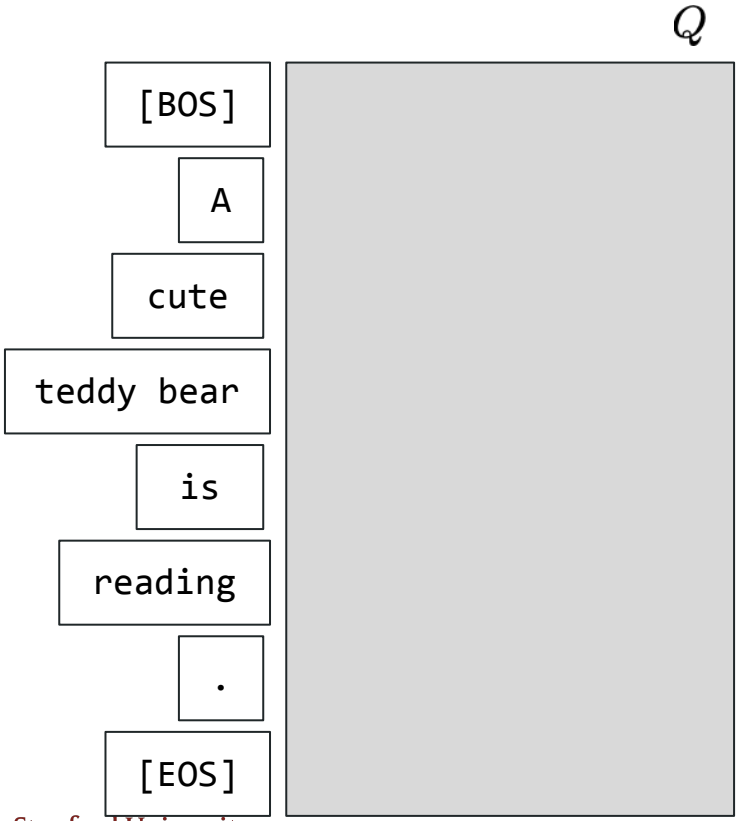
encoder

PAUSE

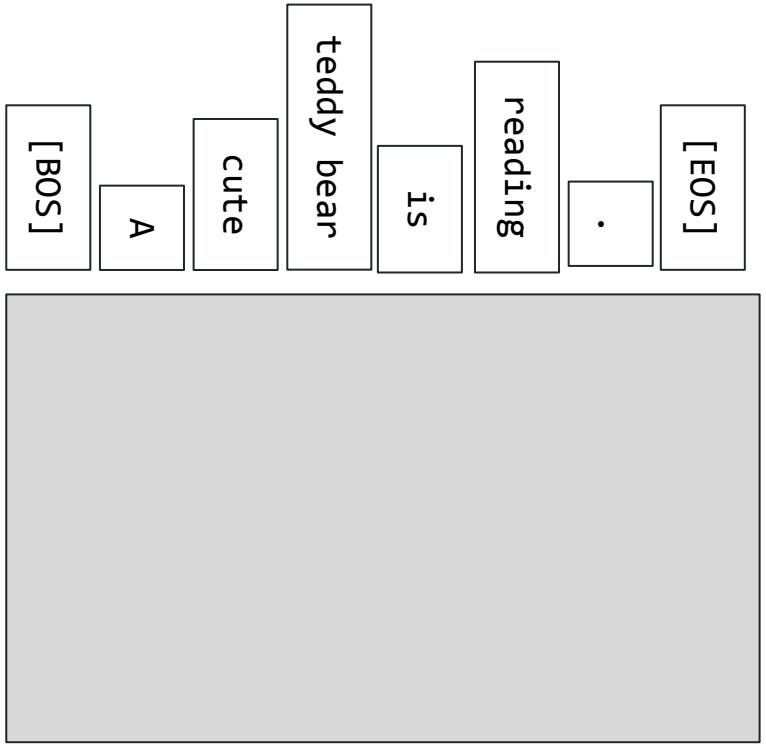
position-aware embeddings matrix

[BOS] A cute teddy bear is reading . [EOS]

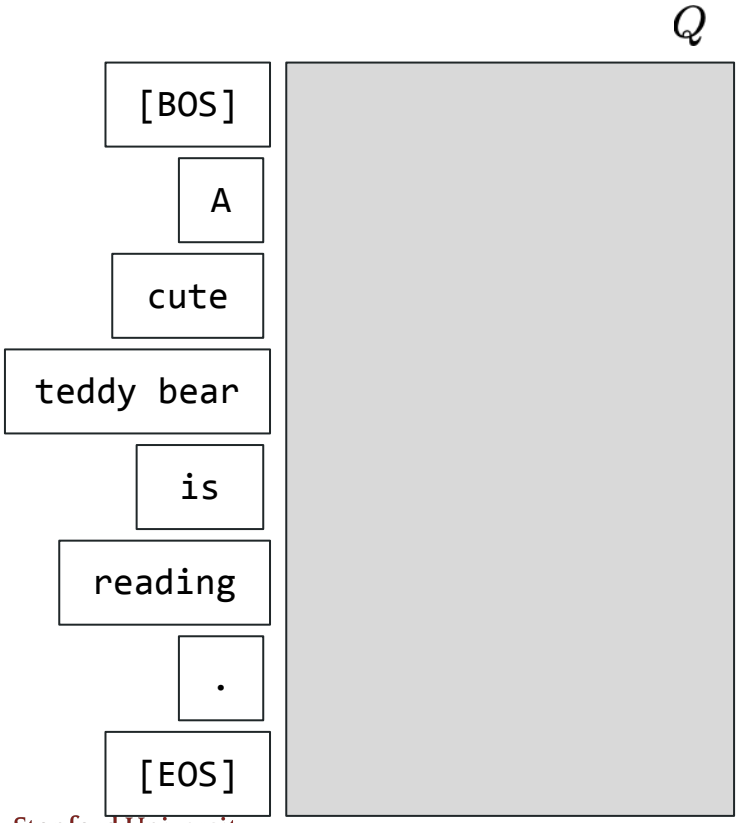
Stitching all the pieces together with an example

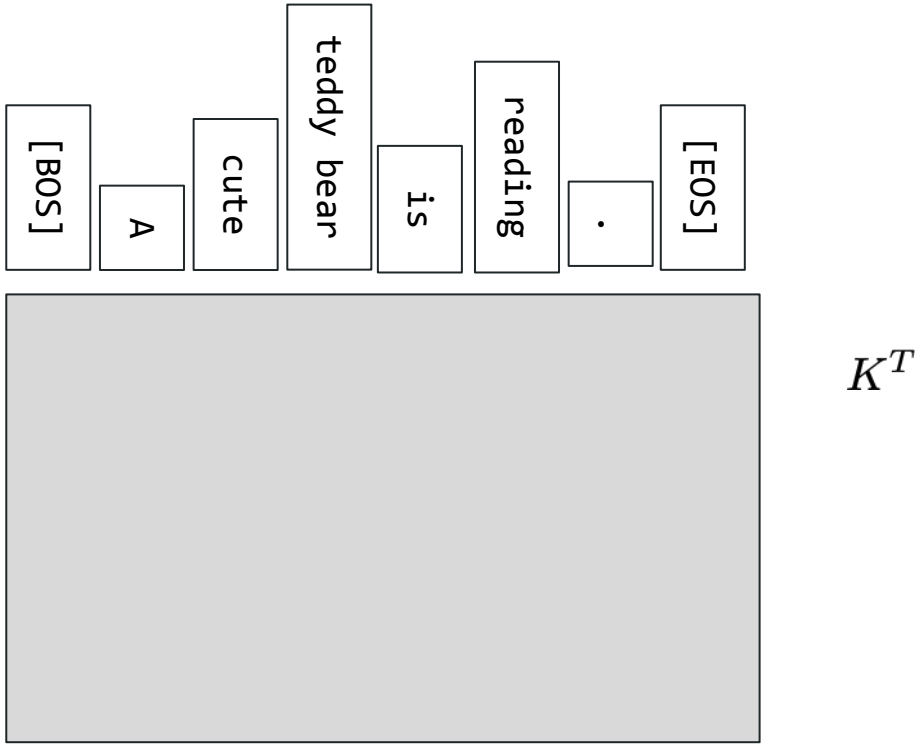


Stitching all the pieces together with an example

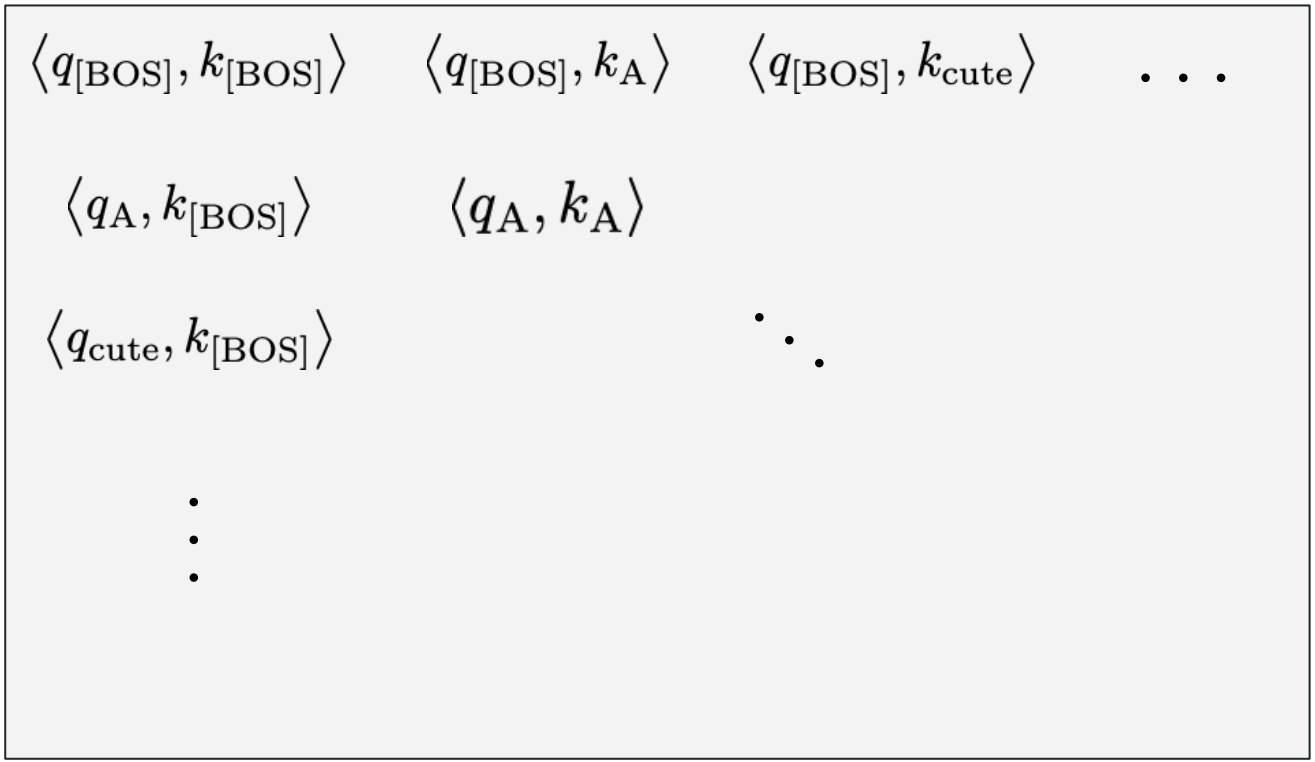


Stitching all the pieces together with an example





Stitching all the pieces together with an example



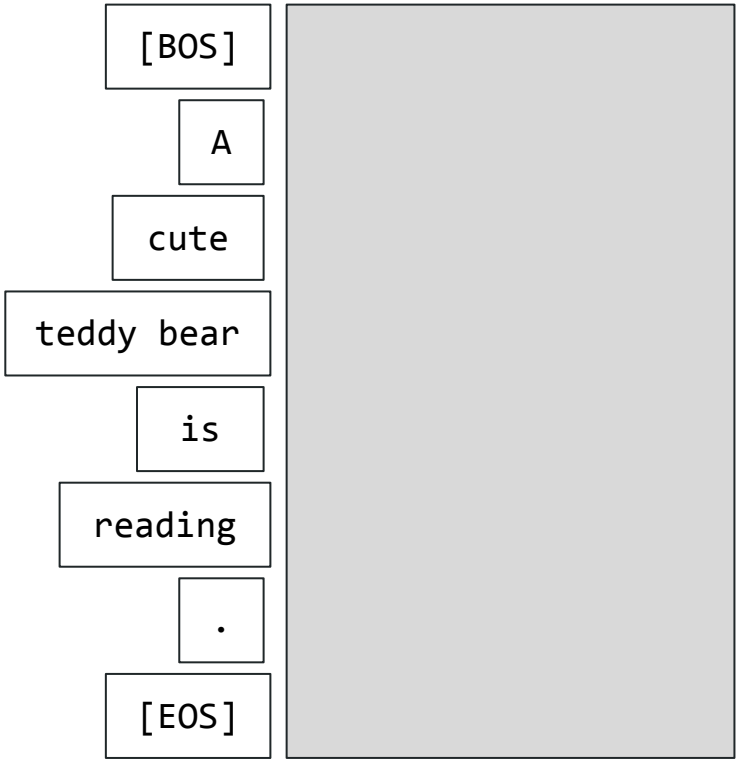
QK^T

Stitching all the pieces together with an example

QK^T

$$\begin{matrix} \langle q_{[\text{BOS}]}, k_{[\text{BOS}]} \rangle & \langle q_{[\text{BOS}]}, k_{\text{A}} \rangle & \langle q_{[\text{BOS}]}, k_{\text{cute}} \rangle & \dots \\ \langle q_{\text{A}}, k_{[\text{BOS}]} \rangle & \langle q_{\text{A}}, k_{\text{A}} \rangle & & \\ \langle q_{\text{cute}}, k_{[\text{BOS}]} \rangle & & \ddots & \\ \vdots & & & \end{matrix}$$

V



Stitching all the pieces together with an example

$$\langle q_{[\text{BOS}]}, k_{[\text{BOS}]} \rangle v_{[\text{BOS}]} + \langle q_{[\text{BOS}]}, k_{\text{A}} \rangle v_{\text{A}} + \langle q_{[\text{BOS}]}, k_{\text{cute}} \rangle v_{\text{cute}} + \dots$$

$$\langle q_{\text{A}}, k_{[\text{BOS}]} \rangle v_{[\text{BOS}]} + \langle q_{\text{A}}, k_{\text{A}} \rangle v_{\text{A}} + \langle q_{\text{A}}, k_{\text{cute}} \rangle v_{\text{cute}} + \dots$$

$$\vdots$$

$$QK^TV$$

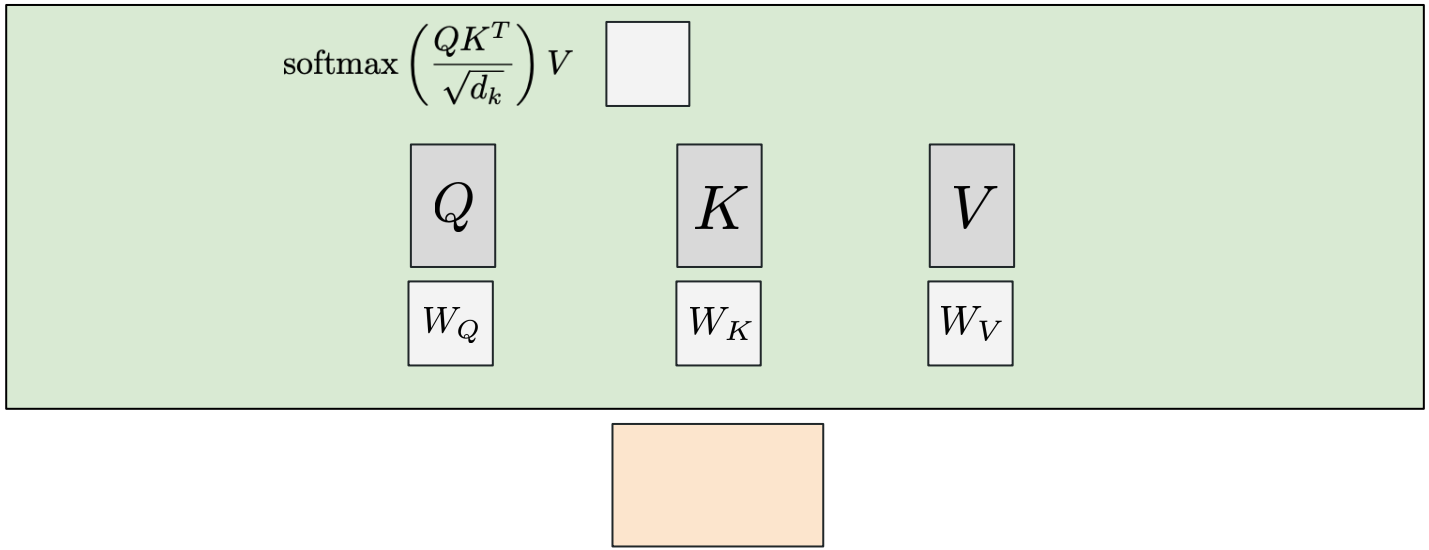
Stitching all the pieces together with an example

weighted average of values

with weights being a function of $\langle q, k \rangle$

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

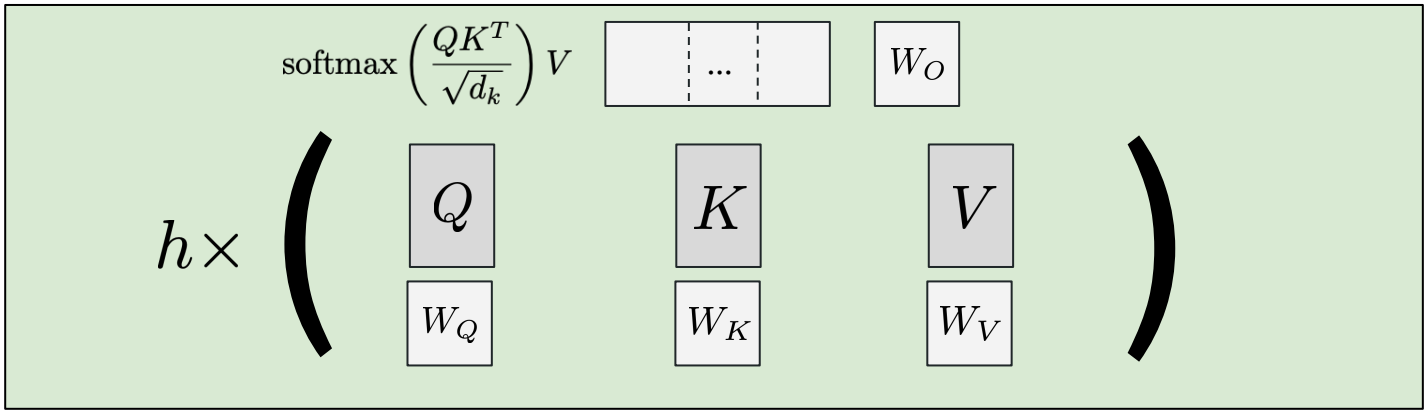
Stitching all the pieces together with an example



encoder

position-aware embeddings matrix

Stitching all the pieces together with an example

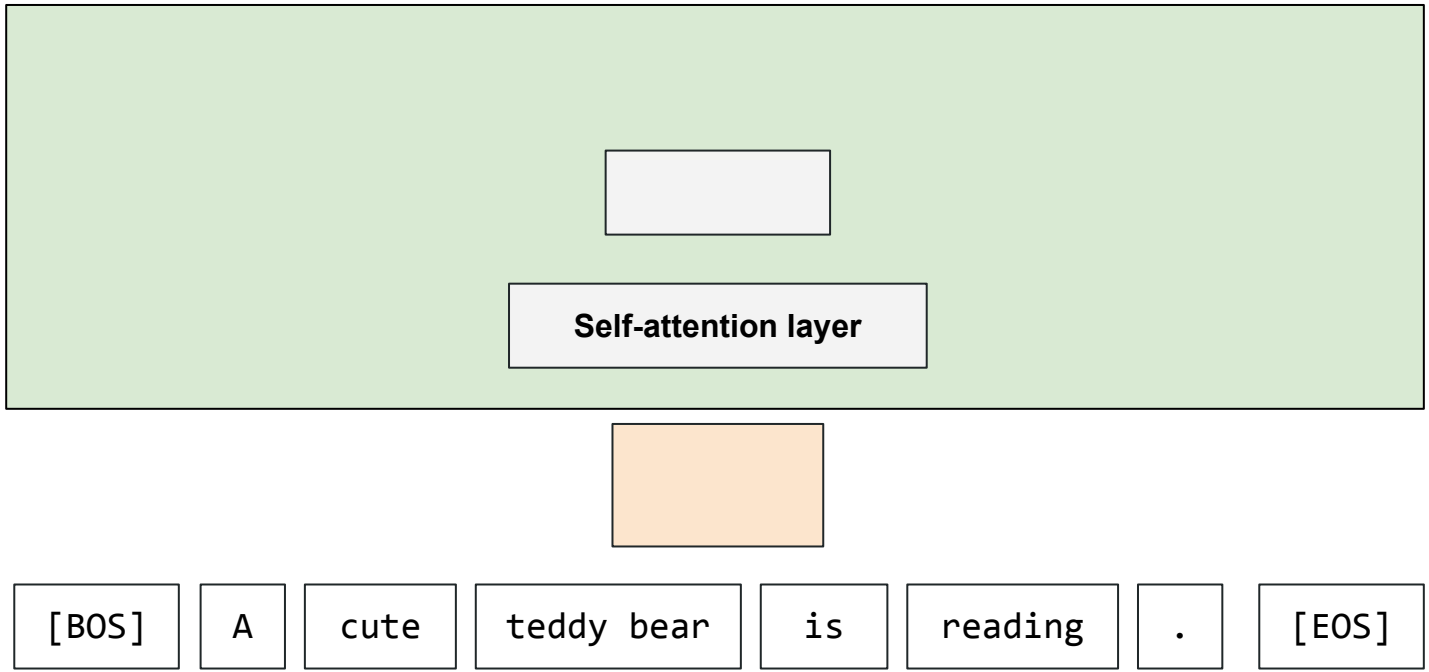


encoder

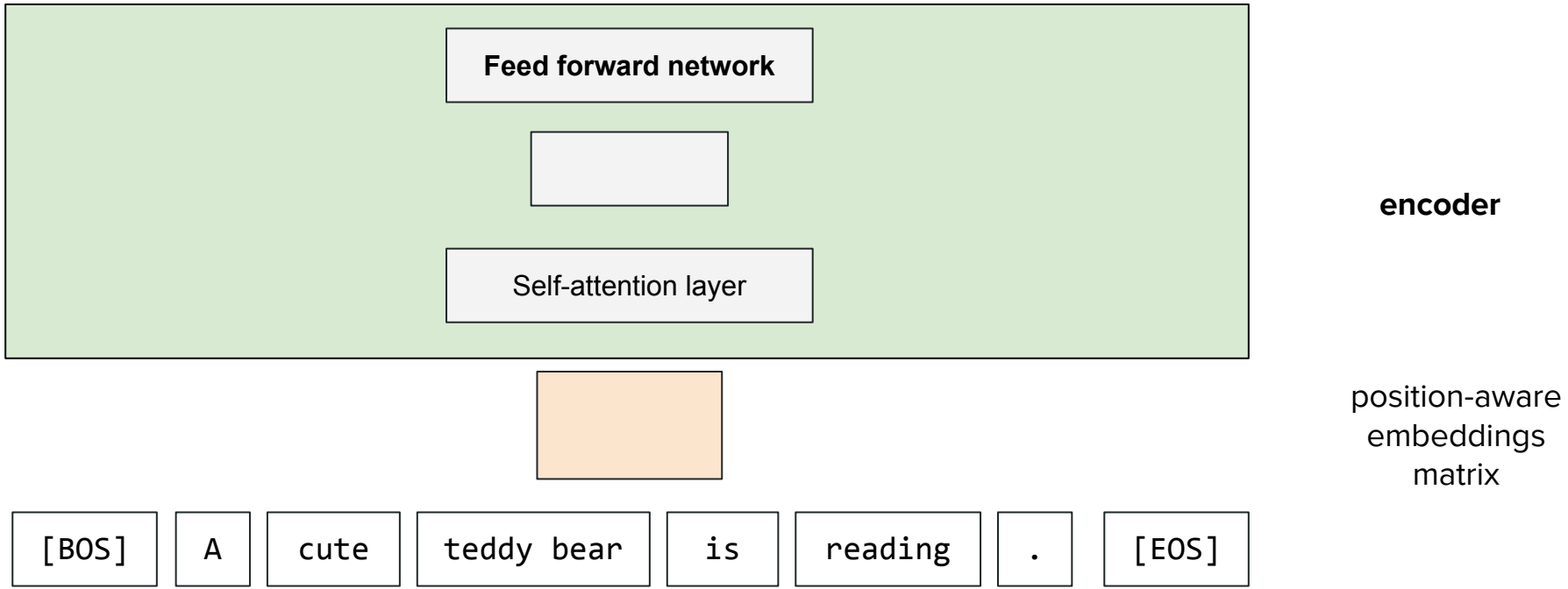
position-aware embeddings matrix

[BOS] A cute teddy bear is reading . [EOS]

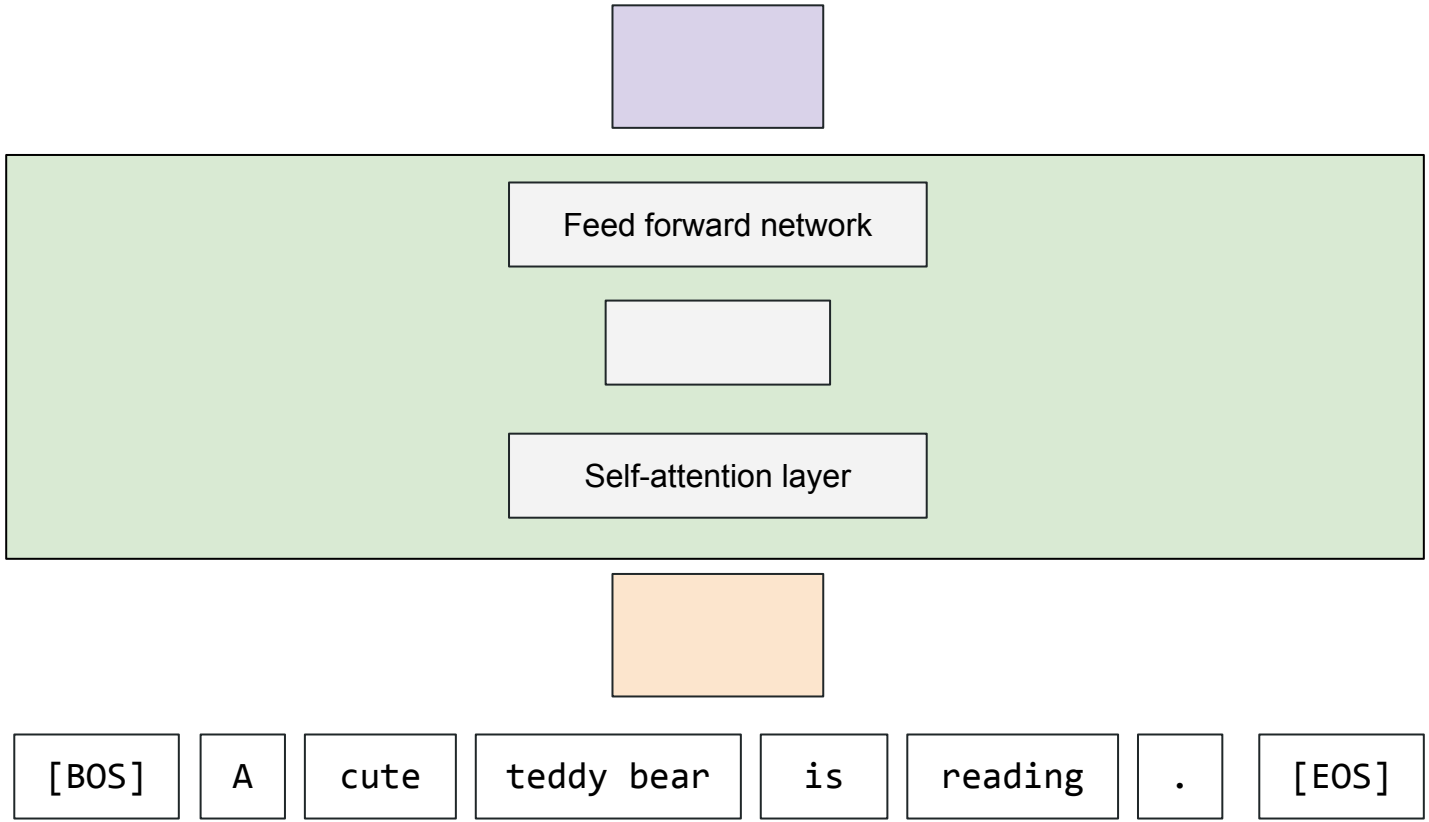
Stitching all the pieces together with an example



Stitching all the pieces together with an example



Stitching all the pieces together with an example

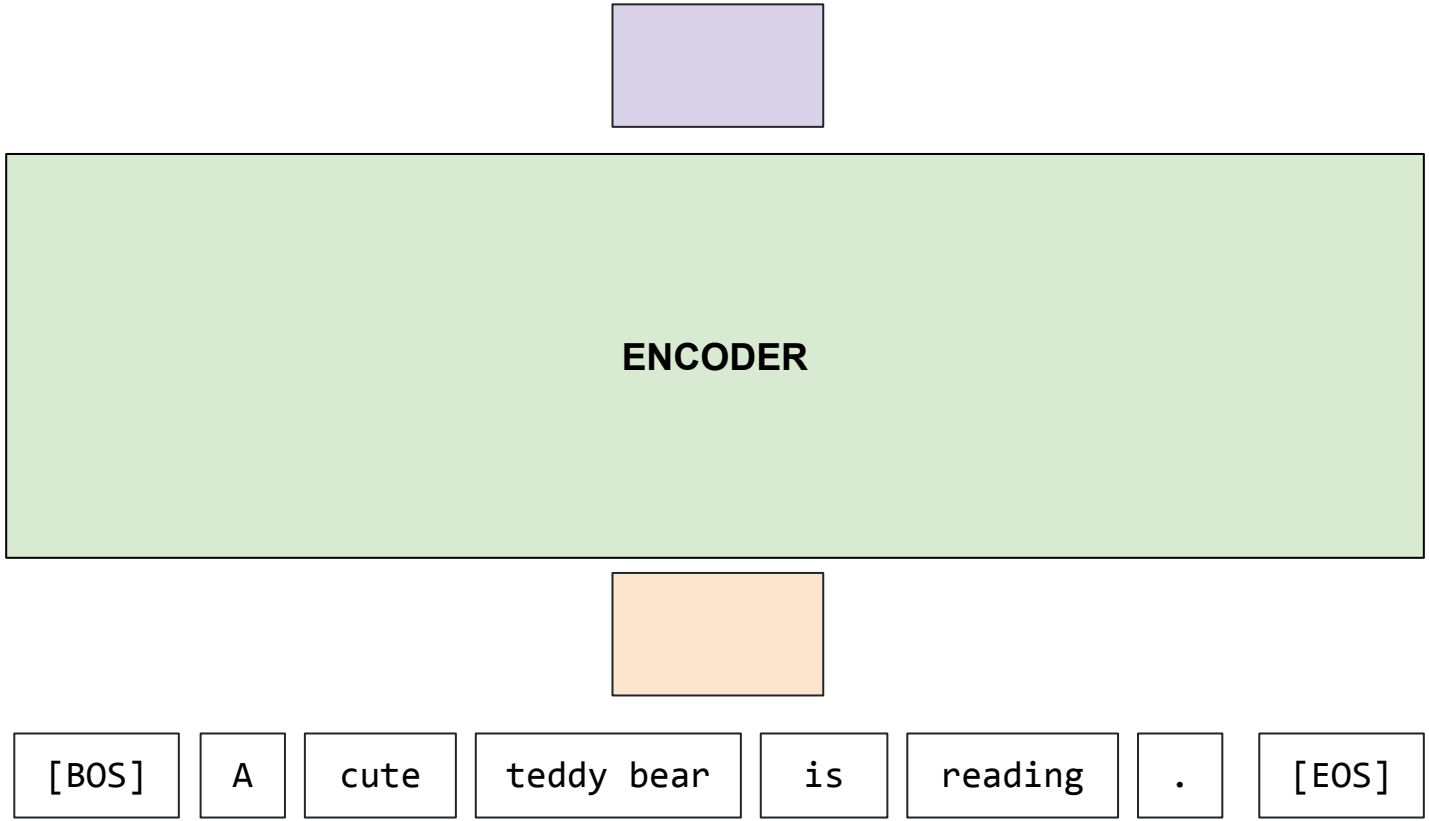


context-aware encoded embeddings

encoder

position-aware embeddings matrix

Stitching all the pieces together with an example



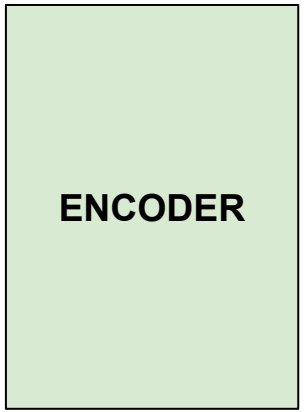
context-aware encoded embeddings

position-aware embeddings matrix

Stitching all the pieces together with an example



encoded
embedding

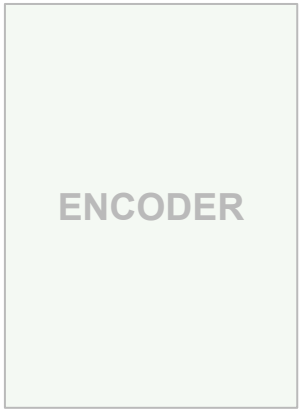



```
A cute teddy bear
   is reading.
```

Stitching all the pieces together with an example

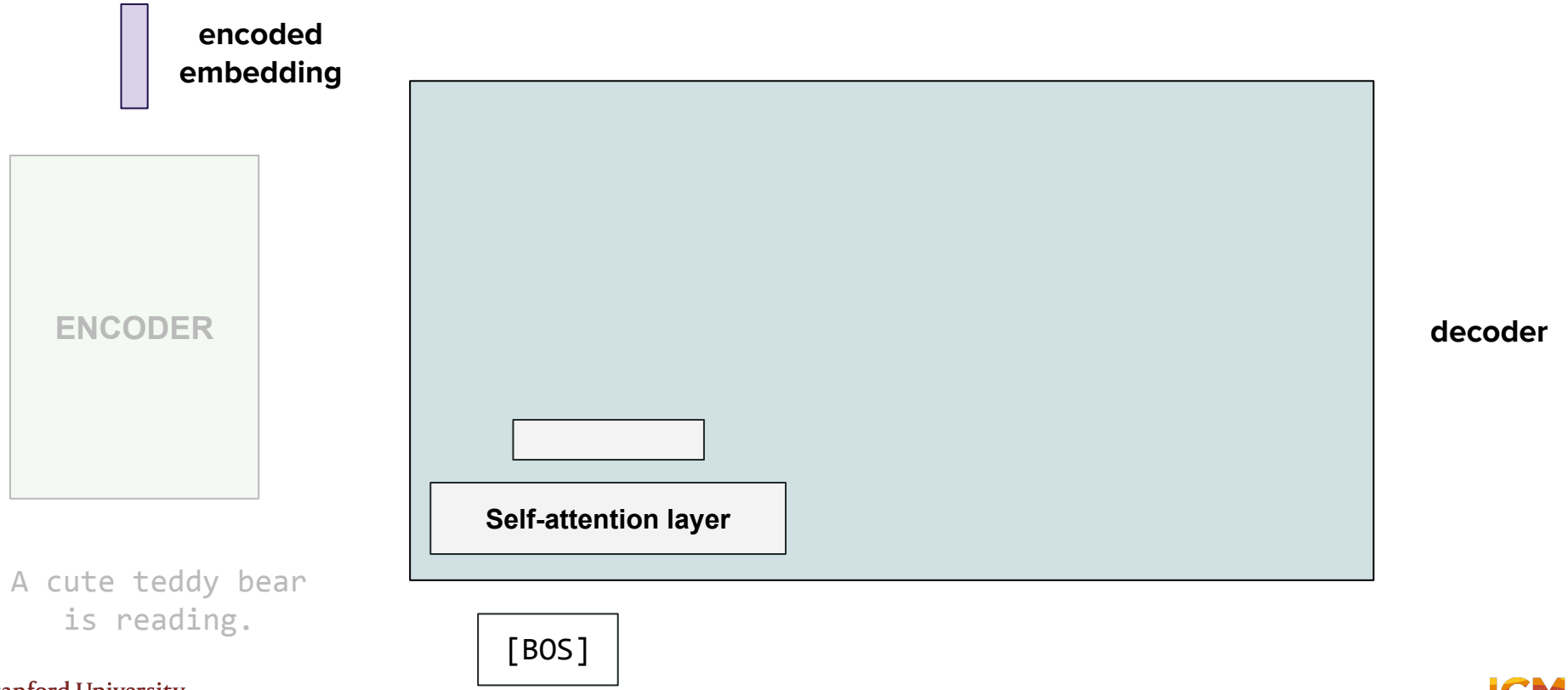


encoded embedding

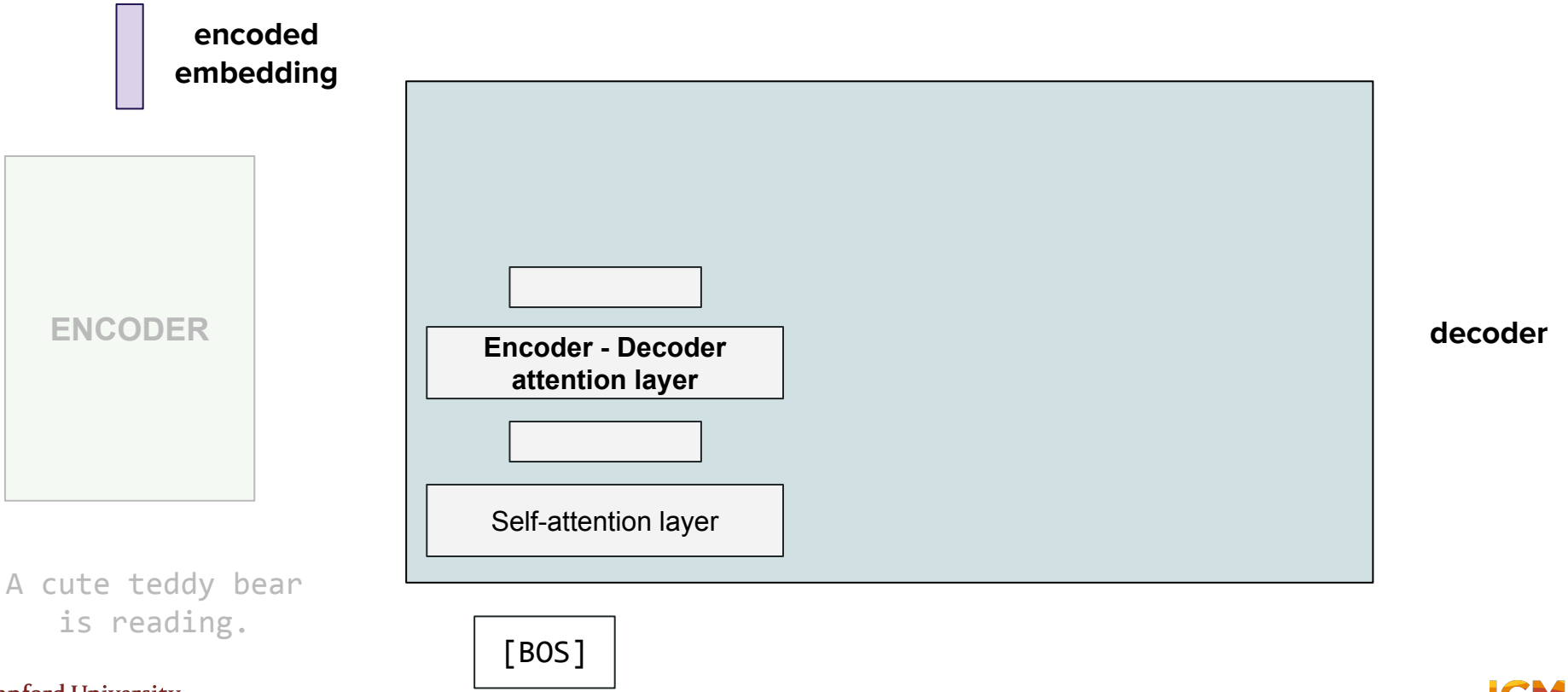


A cute teddy bear is reading.

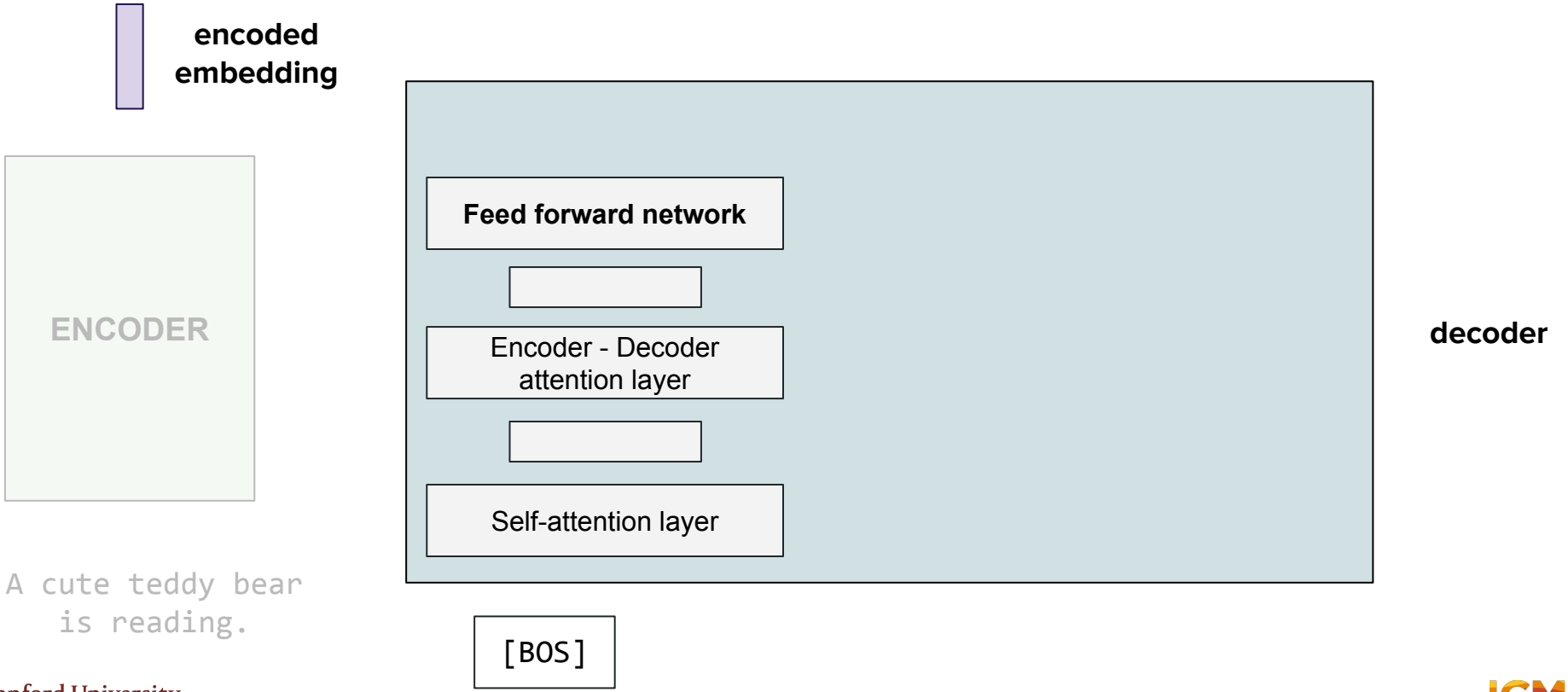
Stitching all the pieces together with an example



Stitching all the pieces together with an example



Stitching all the pieces together with an example



Stitching all the pieces together with an example



encoded embedding

ENCODER

A cute teddy bear is reading.

Softmax layer

Feed forward network

Encoder - Decoder attention layer

Self-attention layer

[BOS]

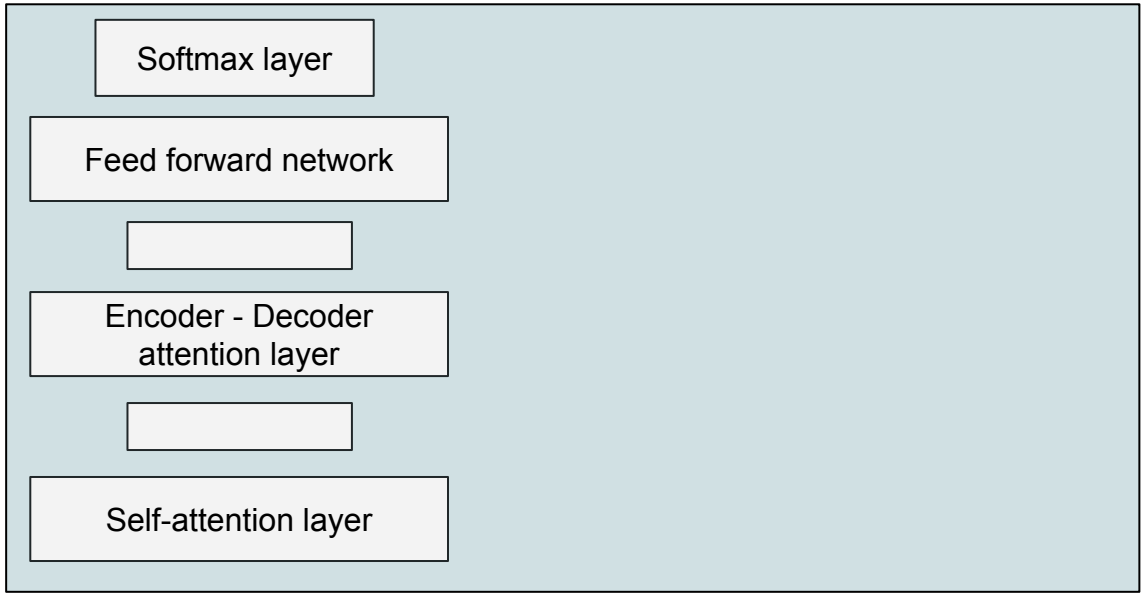
decoder

Stitching all the pieces together with an example



encoded embedding

[0.001, 0.0003, ..., 0.4, ..., 0.002]



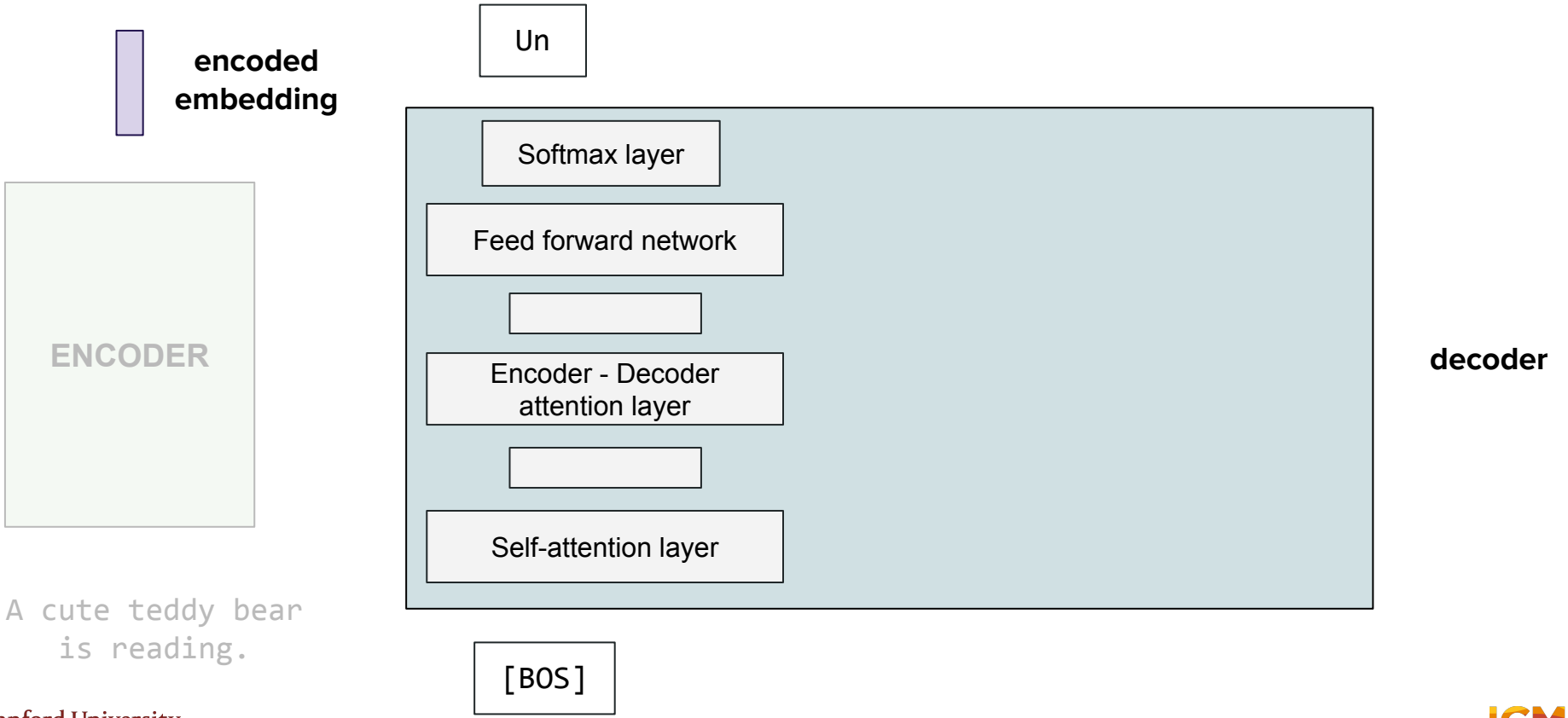
decoder

ENCODER

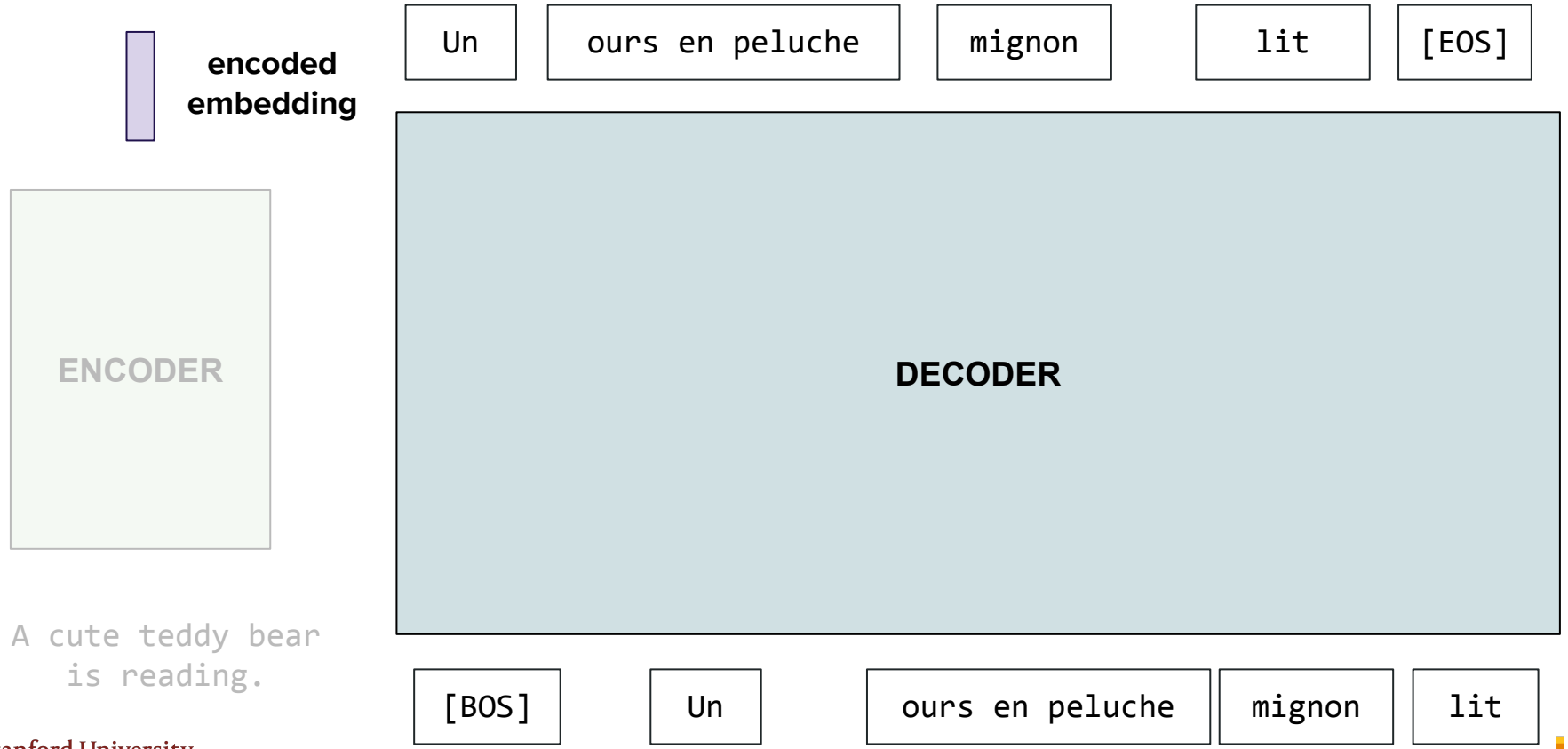
A cute teddy bear is reading.

[BOS]

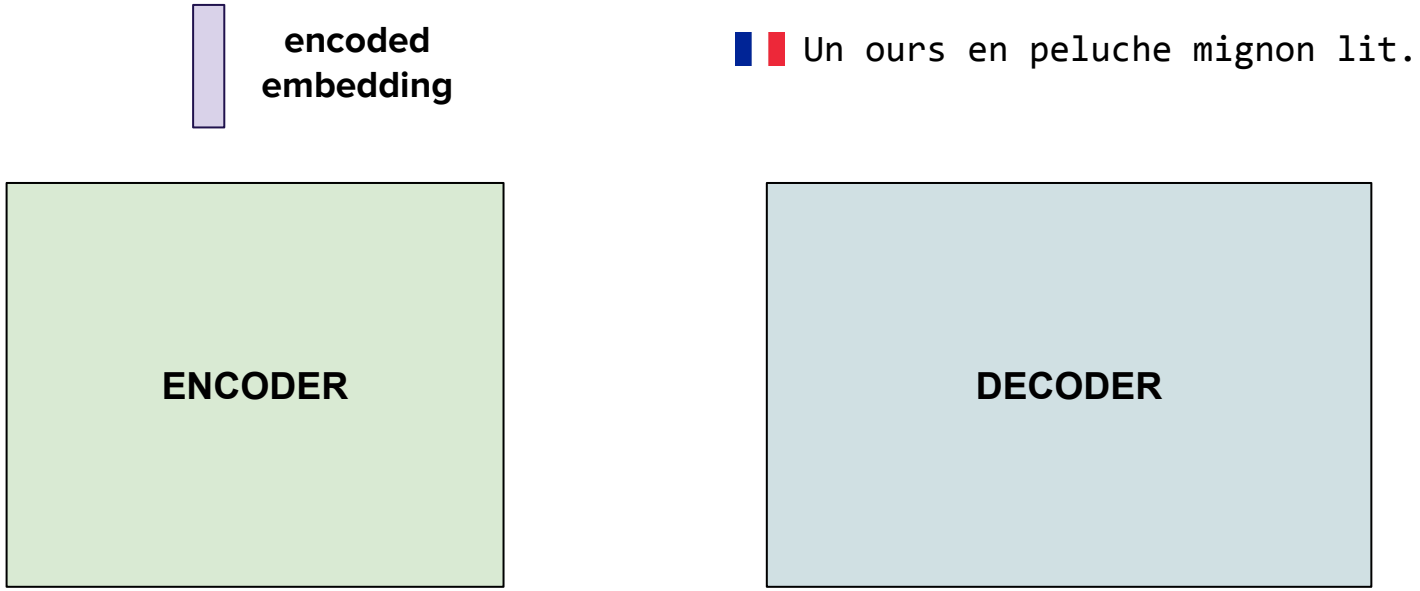
Stitching all the pieces together with an example



Stitching all the pieces together with an example



Stitching all the pieces together with an example





CME 295 Transformers & Large Language Models

Introduction

Tokenization

Word representation

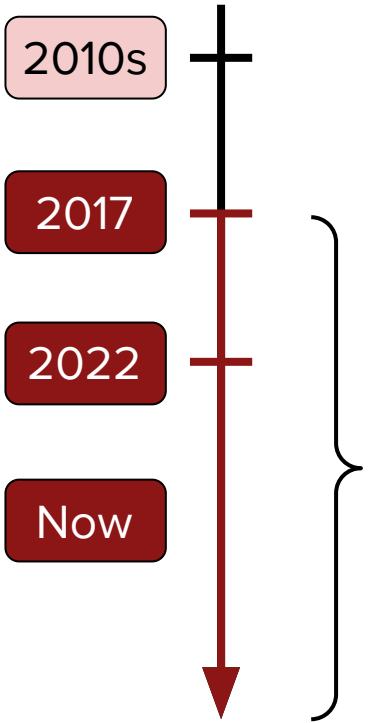
Self-attention mechanism

Transformer architecture

End-to-end example

Discussion

Where to go from here



CME 295 will cover "**conversational**" and "**agentic**" era

Thank you for your attention!